

Package Structure and Notation

The diagram in Figure 22-1 shows a possible package structure for the payroll application. We will address the appropriateness of this structure later. For now, we will confine ourselves to how such a structure is documented and used.

See page 481 for a description of the UML notation for packages. By convention, package diagrams are drawn with the dependencies pointing downwards. Packages at the top are dependent. Packages at the bottom are depended on.

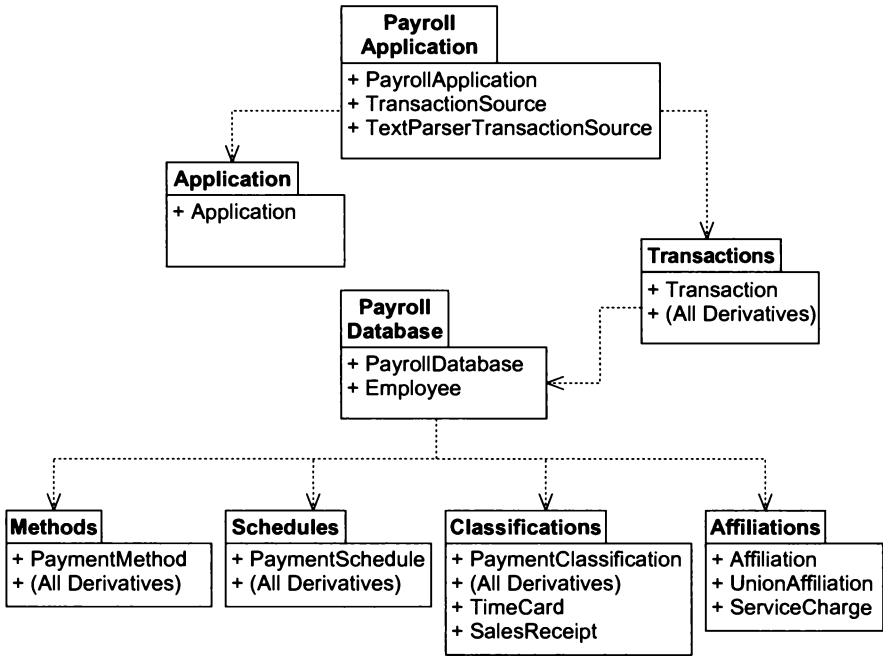


Figure 22-1 Possible Payroll Package Diagram

Figure 22-1 has divided the payroll application into eight packages. The `PayrollApplication` package contains the `PayrollApplication` class and the `TransactionSource` and `TextParserTransactionSource` classes. The `Transactions` package contains the complete `Transaction`-class hierarchy. The constituents of the other packages should be clear by carefully examining the diagram.

The dependencies should also be clear. The `PayrollApplication` package depends on the `Transactions` package because the `PayrollApplication` class calls the `Transaction::Execute` method. The `Transactions` package depends on the `PayrollDatabase` package because each of the many derivatives of `Transaction` communicate directly with the `PayrollDatabase` class. The other dependencies are likewise justifiable.

What criteria did I use to group these classes into packages? I simply stuck the classes that look like they belonged together into the same packages. As we learned in Chapter 20, this is probably not a good idea.

Consider what happens if we make a change to the `Classifications` package. This change will force a recompilation and retest of the `EmployeeDatabase` package, and well it should. But it will also force a recompilation and retest of the `Transactions` package. Certainly the `ChangeClassificationTransaction` and its three derivatives from Figure 19-3 *should* be recompiled and retested, but why should the others be recompiled and retested?

Technically, those other transactions don't need recompilation and retest. However, if they are part of the `Transactions` package, and if the package is going to be rereleased to deal with the changes to the

Classifications package, then it could be viewed as irresponsible not to recompile and retest the package as a whole. Even if all the transactions aren't recompiled and retested, the package itself must be rereleased and re-deployed, and then all of its clients will require revalidation at the very least, and probably recompilation.

The classes in the Transactions package do not share the same closure. Each one is sensitive to its own particular changes. The `ServiceChargeTransaction` is open to changes to the `ServiceCharge` class, whereas the `TimeCardTransaction` is open to changes to the `TimeCard` class. In fact, as the diagram in Figure 22-1 implies, some portion of the Transactions package is dependent on nearly every other part of the software. Thus, this package suffers a very high rate of release. Every time something is changed anywhere below, the Transactions package will have to be revalidated and rereleased.

The `PayrollApplication` package is even more sensitive: any change to any part of the system will affect this package, so its release rate must be enormous. You might think that this is inevitable—that as one climbs higher up the package-dependency hierarchy, the release rate must increase. Fortunately, however, this is not true, and avoiding this symptom is one of the major goals of OOD.

Applying the Common-Closure Principle (CCP)

Consider Figure 22-2. This diagram groups the classes of the payroll application together according to their closure. For example, the `PayrollApplication` package contains the `PayrollApplication` and `TransactionSource` classes. These two classes both depend on the abstract `Transaction` class, which is in the

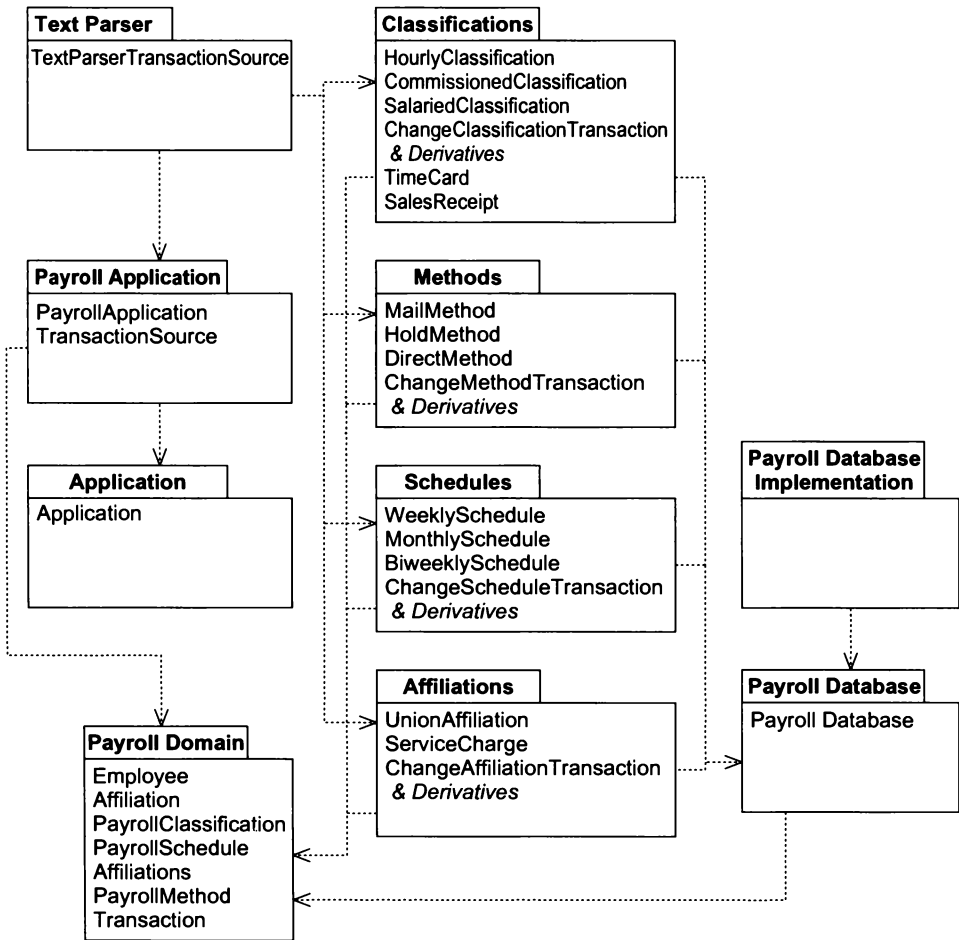


Figure 22-2 A Closed Package Hierarchy for the Payroll Application

PayrollDomain package. Note that the TextParserTransactionSource class is in another package that depends on the abstract PayrollApplication class. This creates an upside-down structure in which the details depend on the generalities, and the generalities are independent. This conforms to the DIP.

The most striking case of generality and independence is the PayrollDomain package. This package contains the *essence* of the whole system, yet it depends on nothing! Examine this package carefully. It contains Employee, PaymentClassification, PaymentMethod, PaymentSchedule, Affiliation, and Transaction. This package contains all of the major abstractions in our model, yet it has no dependencies. Why? Because nearly all of the classes it contains are abstract.

Consider the Classifications package, which contains the three derivatives of PaymentClassification. It also contains the ChangeClassificationTransaction class and its three derivatives, along with TimeCard and SalesReceipt. Notice that any change made to these nine classes is isolated; other than TextParser, no other package is affected! Such isolation also holds for the Methods package, the Schedules package, and the Affiliations package. This is quite a bit of isolation.

Notice that the bulk of the executable code is in packages that have few or no dependents. Since almost nothing depends on them, we call them *irresponsible*. The code within those packages is tremendously flexible; it can be changed without affecting many other parts of the project. Notice also that the most general packages of the system contain the least amount of executable code. These packages are heavily depended on, but depend on nothing. Since many packages depend on them, we call them *responsible*, and since they don't depend on anything, we call them *independent*. Thus, the amount of responsible code (i.e., code in which changes would affect lots of other code) is very small. Moreover, that small amount of responsible code is also independent, which means that no other modules will induce it to change. This upside-down structure, with highly independent and responsible generalities at the bottom, and highly irresponsible and dependent details at the top, is the hallmark of object-oriented design.

Contrast Figure 22-1 with Figure 22-2. Notice that the details at the bottom of Figure 22-1 are independent and highly responsible. This is the wrong place for details! Details should depend on the major architectural decisions of the system and should not be depended on. Notice also that the generalities, the packages that define the architecture of the system, are irresponsible and highly dependent. Thus, the packages that define the architectural decisions depend on, and are thus constrained by, the packages that contain the implementation details. This is a violation of the SAP. It would be better if the architecture constrained the details!

Applying the Reuse–Release Equivalency Principle (REP)

What portions of the payroll application can we reuse? If another division of our company wanted to reuse our payroll system, but they had a completely different set of policies, they could not reuse Classifications, Methods, Schedules, or Affiliations. However, they could reuse PayrollDomain, PayrollApplication, Application, PayrollDatabase, and possibly PDImplementation. On the other hand, if another department wanted to write software that analyzed the current employee database, they could reuse PayrollDomain, Classifications, Methods, Schedules, Affiliations, PayrollDatabase, and PDImplementation. In each case, the granule of reuse is a package.

Seldom, if ever, would only a single class from a package be reused. The reason is simple: the classes within a package should be cohesive. That means that they depend on one another and cannot be easily or sensibly separated. It would make no sense, for example, to use the Employee class without using the PaymentMethod class. In fact, in order to do so, you would have to modify the Employee class so that it did not contain a PaymentMethod class. Certainly we don't want to support the kind of reuse that forces us to modify the reused components. Therefore, the granule of reuse is the package. This gives us another cohesion criterion to employ when trying to group classes into packages: not only should the classes be closed together, they should also be reusable together in conformance with the REP.

Consider again our original package diagram in Figure 22-1. The packages that we might like to reuse, like `Transactions` or `PayrollDatabase`, are not easily reusable because they drag along a lot of extra baggage. The `PayrollApplication` package is horribly dependent (it depends on everything). If we wanted to create a new payroll application that used a different set of schedule, method, affiliation, and classification policies, we would not be able to use this package as a whole. Instead, we would have to take individual classes from `PayrollApplication`, `Transactions`, `Methods`, `Schedules`, `Classifications`, and `Affiliations`. By disassembling the packages in this way, we destroy their release structure. We cannot say that Release 3.2 of `PayrollApplication` is reusable.

Figure 22-1 violates the CRP. Thus, having accepted the reusable fragments of our various packages, the reuser will be faced with a difficult management problem: he will not be able to depend on our release structure. A new release of `Methods` affects him because he is reusing the `PaymentMethod` class. Most of the time, the changes will be to classes that he is not reusing, yet he must still track our new release number and probably recompile and retest his code.

This can be so difficult to manage that the reuser's most likely strategy will be to make a copy of the reusable components and evolve that copy separately from ours. This is not reuse. The two pieces of code will become different and will require independent support, effectively doubling the support burden.

These problems are not exhibited by the structure in Figure 22-2. The packages in that structure are easier to reuse. `PayrollDomain` does not drag along much baggage. It is reusable independently of any of the derivatives of `PaymentMethod`, `PaymentClassification`, `PaymentSchedule`, etc.

The astute reader will notice that the package diagram in Figure 22-2 does not completely conform to the CRP. Specifically, the classes within `PayrollDomain` do not form the smallest reusable unit. The `Transaction` class does not need to be reused with the rest of the package. We could design many applications that access the `Employee` and its fields, but never use a `Transaction`.

This suggests a change to the package diagram, as shown in Figure 22-3. This separates the transactions from the elements that they manipulate. For example, the classes in the `MethodTransactions` package manipulate the classes in the `Methods` package. We have moved the `Transaction` class into a new package named `TransactionApplication`, which also contains `TransactionSource` and a class named `TransactionApplication`. These three form a reusable unit. The `PayrollApplication` class has now become the grand unifier. It contains the main program and also a derivative of `TransactionApplication` called `PayrollApplication`, which ties the `TextParserTransactionSource` to the `TransactionApplication`.

These manipulations have added yet another layer of abstraction to the design. The `TransactionApplication` package can now be reused by any application that obtains `Transactions` from a `TransactionSource` and then Executes them. The `PayrollApplication` package is no longer reusable, since it is extremely dependent. However, the `TransactionApplication` package has taken its place, and is more general. Now we can reuse the `PayrollDomain` package without any `Transactions`.

This certainly improves the reusability and maintainability of the project, but at the cost of five extra packages and a more complex dependency architecture. The value of the trade-off depends on the type of reuse that we might expect and the rate at which we expect the application to evolve. If the application remains very stable, and few clients reuse it, then this change is overkill. On the other hand, if many applications will reuse this structure, or we expect the application to experience many changes, then the new structure is superior—it's a judgment call; and it should be driven by data rather than speculation. It is best to start simple and grow the package structure as necessary. Package structures can always be made more elaborate if necessary.

Coupling and Encapsulation

Just as the coupling among classes is managed by encapsulation boundaries in Java and C++, so the couplings among packages can be managed by the export adornments of the UML.

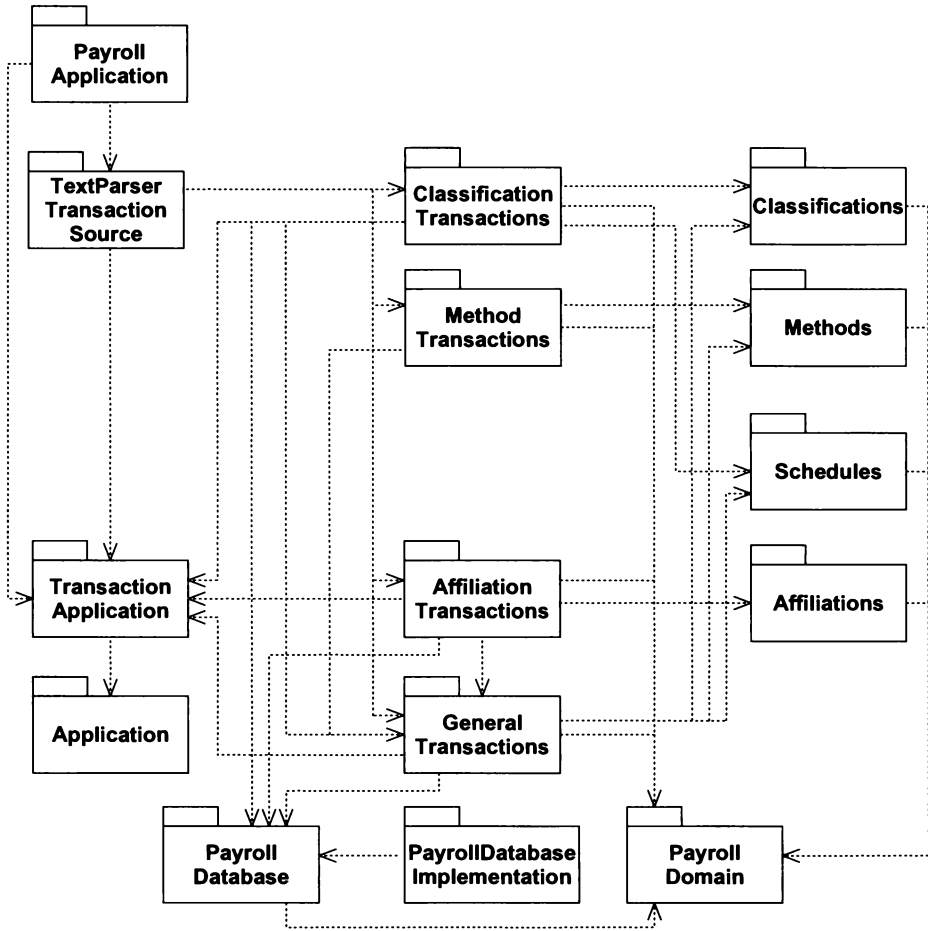


Figure 22-3 Updated Payroll Package Diagram

If a class within one package is to be used by another package, that class must be exported. In UML, classes are exported by default, but we may adorn a package to denote that certain classes should not be exported. Figure 22-4, a blowup of the *Classifications* package, shows that the three derivatives of *PaymentClassification* are exported, but that *TimeCard* and *SalesReceipt* are not. This means that other packages will not be able to use *TimeCard* and *SalesReceipt*; they are private to the *Classifications* package.

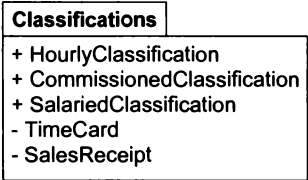


Figure 22-4 Private Classes in Classifications Package

We may want to hide certain classes within a package to prevent afferent couplings. *Classifications* is a very detailed package that contains the implementations of several payment policies. In order to keep this package on the main sequence, we want to limit its afferent couplings, so we hide the classes that other packages don't need to know about.

TimeCard and SalesReceipt are good choices for private classes. They are implementation details of the mechanisms for calculating an employee’s pay. We want to remain free to alter these details, so we need to prevent anyone else from depending on their structure.

A quick glance at Figure 19-7 through Figure 19-10 and Listing 19-15 (page 213 through page 217) shows that the TimeCardTransaction and SalesReceiptTransaction classes already depend on TimeCard and SalesReceipt. We can easily resolve this problem, however, as shown in Figure 22-5 and Figure 22-6.

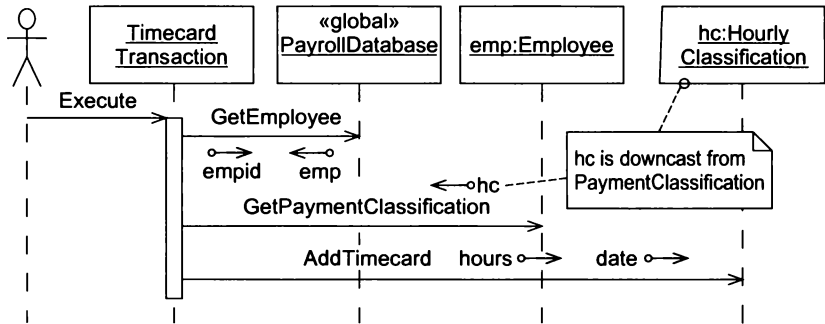


Figure 22-5 Revision to TimeCardTransaction to Protect TimeCard Privacy

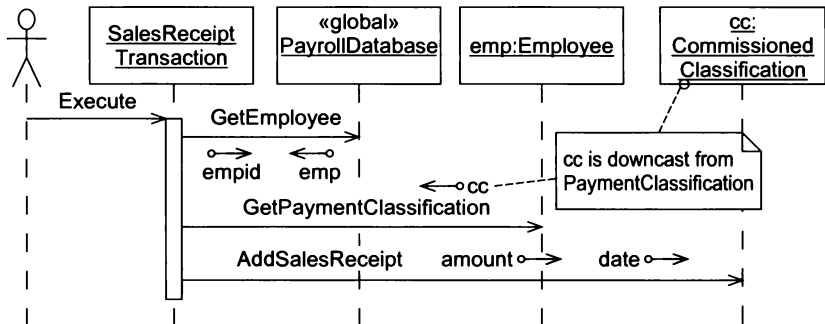


Figure 22-6 Revision to SalesReceiptTransaction to Protect SalesReceipt Privacy

Metrics

As we showed in Chapter 20, we can quantify the attributes of cohesion, coupling, stability, generality, and conformance to the main sequence with a few simple metrics. But why should we want to? To paraphrase Tom DeMarco: You can’t manage what you can’t control, and you can’t control what you don’t measure.¹ To be effective software engineers or software managers, we must be able to control software development practice. If we don’t measure it, however, we will never have that control.

By applying the heuristics described below and calculating some fundamental metrics about our object-oriented designs, we can begin to correlate those metrics with measured performance of the software and of the teams that develop it. The more metrics we gather, the more information we will have, and the more control we will eventually be able to exert.

The metrics below have been successfully applied to a number of projects since 1994. There are several automatic tools that will calculate them for you, and they are not hard to calculate by hand. It is also not hard to write a simple shell, python, or ruby script to walk through your source files and calculate them.²

1. [DeMarco82], p. 3.
2. For an example of a shell script, you can download `depend.sh` from the freeware section of www.objectmentor.com, or take a look at JDepend at www.clarkware.com.

- **(H) Relational Cohesion.** One aspect of the cohesion of a package can be represented as the average number of internal relationships per class. Let R be the number of class relationships that are internal to the package (i.e., that do not connect to classes outside the package). Let N be the number of classes within the package). The extra 1 in the formula prevents $H = 0$ when $N = 1$. It represents the relationship that the package has to all its classes.

$$H = \frac{R + 1}{N}$$

- **(C_a) Afferent coupling** can be calculated as the number of classes from other packages that depend on the classes within the subject package. These dependencies are class relationships, such as inheritance and association.
- **(C_e) Efferent coupling** can be calculated as the number of classes in other packages that the classes in the subject package depend on. As before, these dependencies are class relationships.
- **(A) Abstractness or Generality** can be calculated as the ratio of the number of abstract classes (or interfaces) in the package to the total number of classes (and interfaces) in the package.³ This metric ranges from 0 to 1.

$$A = \frac{\text{Abstract Classes}}{\text{Total Classes}}$$

- **(I) Instability** can be calculated as the ratio of efferent coupling to total coupling. This metric also ranges from 0 to 1.

$$I = \frac{C_e}{C_e + C_a}$$

- **(D) Distance from the Main Sequence.** The main sequence is idealized by the line $A + I = 1$. The formula for D calculates the distance of any particular package from the main sequence. It ranges from $\sim .7$ to 0;⁴ the closer to 0, the better.

$$D = \frac{|A + I - 1|}{\sqrt{2}}$$

- **(D') Normalized Distance from the Main Sequence.** This metric represents the D metric normalized to the range [0,1]. It is perhaps a little more convenient to calculate and to interpret. The value 0 represents a package that is coincident with the main sequence. The value 1 represents a package that is as far from the main sequence as is possible.

$$D' = |A + I - 1|$$

Applying the Metrics to the Payroll Application

Table 22-1 shows how the classes in the payroll model have been allocated to packages. Figure 22-7 shows the package diagram for the payroll application with all the metrics calculated. And Table 22-2 shows all of the metrics calculated for each package.

- One might think that a better formula for A is the ratio of pure virtual functions to total member functions within the package. However, I have found that this formula weakens the abstraction metric too much. Even one pure virtual function will make a class abstract, and the power of that abstraction is more significant than the fact that the class may have dozens of concrete functions, especially when the DIP is being followed.
- It is impossible to plot any package outside the unit square on the graph of A vs. I . This is because neither A nor I can exceed 1. The main sequence bisects this square from (0,1) to (1,0). The points within the square that are farthest from the main sequence are the two corners (0,0) and (1,1). Their distance from the main sequence is

$$\frac{\sqrt{2}}{2} = 0.70710678\dots$$

Each package dependency in Figure 22-7 is adorned with two numbers. The number closest to the depender package represents the number of classes in that package that depend on the dependee package. The number closest to the dependee package represents the number of classes in that package that the depender package depends on.

Table 22-1

Package	Classes in Package		
Affiliations	ServiceCharge	UnionAffiliation	
AffiliationTransactions	ChangeAffiliationTransaction	ChangeUnaffiliated-Transaction	ChangeMember-Transaction
	ServiceChargeTransaction		
Application	Application		
Classifications	CommissionedClassification	HourlyClassification	SalariedClassification
	SalesReceipt	Timecard	
ClassificationTransaction	ChangeClassification-Transaction	ChangeCommissioned-Transaction	ChangeHourly-Transaction
	ChangeSalariedTransaction	SalesReceiptTransaction	TimecardTransaction
GeneralTransactions	AddCommissionedEmployee	AddEmployeeTransaction	AddHourlyEmployee
	AddSalariedEmployee	ChangeAddressTransaction	ChangeEmployee-Transaction
	ChangeNameTransaction	DeleteEmployeeTransaction	PaydayTransaction
Methods	DirectMethod	HoldMethod	MailMethod
MethodTransactions	ChangeDirectTransaction	ChangeHoldTransaction	ChangeMailTransaction
	ChangeMethodTransaction		
PayrollApplication	PayrollApplication		
PayrollDatabase	PayrollDatabase		
PayrollDatabase-Implementation	PayrollDatabase-Implementation		
PayrollDomain	Affiliation	Employee	PaymentClassification
	PaymentMethod	PaymentSchedule	
Schedules	BiweeklySchedule	MonthlySchedule	WeeklySchedule
TextParserTransaction-Source	TextParserTransactionSource		
TransactionApplication	TransactionApplication	Transaction	TransactionSource

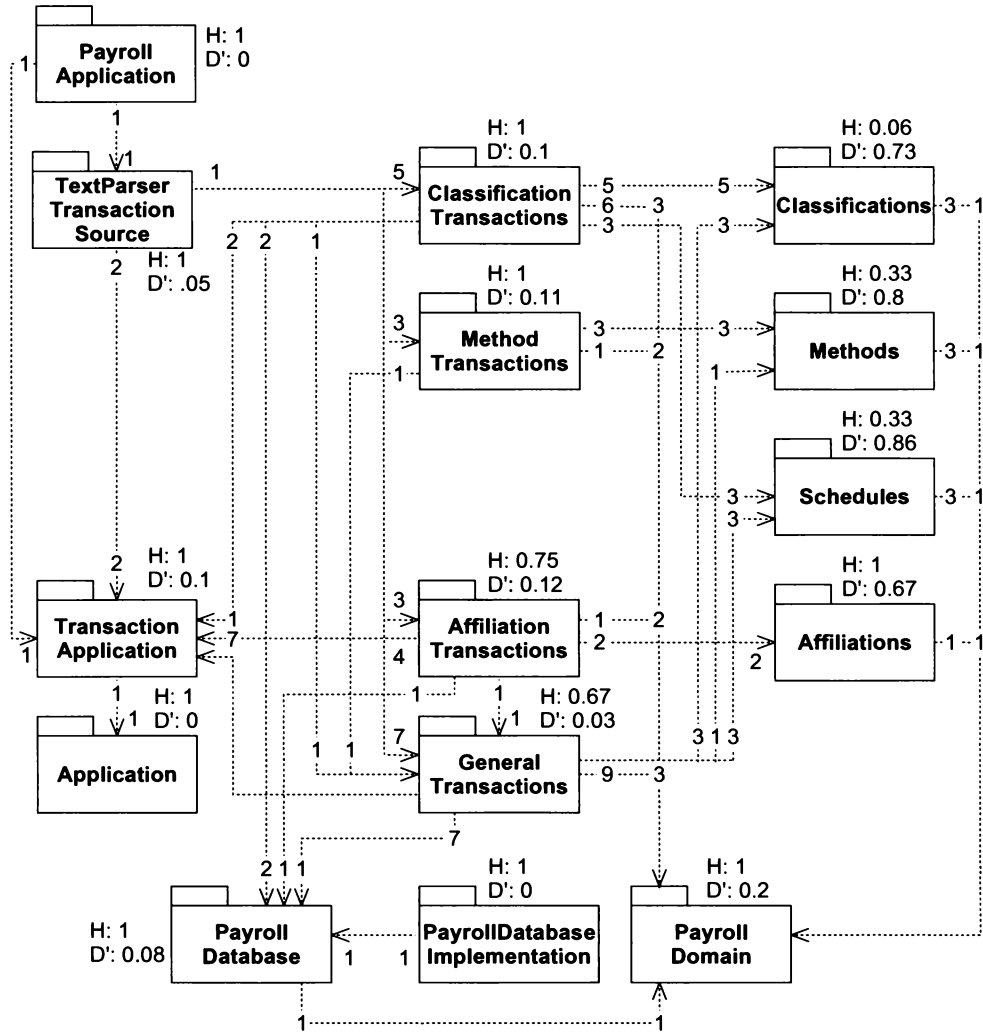


Figure 22-7 Package Diagram with Metrics

Each package in Figure 22-7 is adorned with the metrics that apply to it. Many of these metrics are encouraging. `PayrollApplication`, `PayrollDomain`, and `PayrollDatabase`, for example, have high relational cohesion and are either on or close to the main sequence. However, the `Classifications`, `Methods`, and `Schedules` packages show generally poor relational cohesion and are almost as far from the main sequence as is possible!

These numbers tell us that the partitioning of the classes into packages is weak. If we don't find a way to improve the numbers, then the development environment will be sensitive to change, which may cause unnecessary rerelease and retesting. Specifically, we have low-abstraction packages like `ClassificationTransactions` depending heavily on other low-abstraction packages like `Classifications`. Classes with low abstraction contain most of the detailed code and are therefore likely to change, which will force rerelease of the packages that depend on them. Thus the `ClassificationTransactions` package will have a very high release rate since it is subject to both its own high change rate and that of `Classifications`. As much as possible, we would like to limit the sensitivity of our development environment to change.

Clearly, if we have only two or three developers, they will be able to manage the development environment “in their heads,” and the need to maintain packages on the main sequence, for this purpose, will not be great. The

Table 22-2

Package Name	N	A	Ca	Ce	R	H	I	A	D	D'
Affiliations	2	0	2	1	1	1	.33	0	.47	.67
AffiliationTransactions	4	1	1	7	2	.75	.88	.25	.09	.12
Application	1	1	1	0	0	1	0	1	0	0
Classifications	5	0	8	3	2	.06	.27	0	.51	.73
ClassificationTransaction	6	1	1	14	5	1	.93	.17	.07	.10
GeneralTransactions	9	2	4	12	5	.67	.75	.22	.02	.03
Methods	3	0	4	1	0	.33	.20	0	.57	.80
MethodTransactions	4	1	1	6	3	1	.86	.25	.08	.11
PayrollApplication	1	0	0	2	0	1	1	0	0	0
PayrollDatabase	1	1	11	1	0	1	.08	1	.06	.08
PayrollDatabaseImpl...	1	0	0	1	0	1	1	0	0	0
PayrollDomain	5	4	26	0	4	1	0	.80	.14	.20
Schedules	3	0	6	1	0	.33	.14	0	.61	.86
TextParserTransactionSource	1	0	1	20	0	1	.95	0	.03	.05
TransactionApplication	3	3	9	1	2	1	.1	1	.07	.10

more developers there are, however, the more difficult it is to keep the development environment sane. Moreover, the work required to obtain these metrics is minimal compared to the work required to do even a single retest and rerelease.⁵ Therefore, it is a judgment call as to whether the work of computing these metrics will be a short-term loss or gain.

Object Factories

Classifications and ClassificationTransactions are so heavily depended on because the classes within them must be instantiated. For example, the TextParserTransactionSource class must be able to create AddHourlyEmployeeTransaction objects; thus, there is an afferent coupling from the TextParserTransactionSource package to the ClassificationTransactions package. Also, the ChangeHourlyTransaction class must be able to create HourlyClassification objects, so there is an afferent coupling from the ClassificationTransactions package to the Classification package.

Almost every other use of the objects within these packages is through their abstract interface. Were it not for the need to create each concrete object, the afferent couplings on these packages would not exist. For example, if TextParserTransactionSource did not need to create the different transactions, it would not depend on the four packages containing the transaction implementations.

This problem can be significantly mitigated by using the FACTORY pattern. Each package provides an object factory that is responsible for creating all the public objects within that package.

5. I spent about two hours compiling by hand the statistics and computing the metrics for the payroll example. Had I used one of the commonly available tools, it would have taken virtually no time at all.

The Object Factory for TransactionImplementation

Figure 22-8 shows how to build an object factory for the TransactionImplementation package. The TransactionFactory package contains the abstract base class, which defines the pure virtual functions that represent the constructors for the concrete transaction objects. The TransactionImplementation package contains the concrete derivative of the TransactionFactory class and uses all the concrete transactions in order to create them.

The TransactionFactory class has a static member declared as a TransactionFactory pointer. This member must be initialized by the main program to point to an instance of the concrete TransactionFactoryImplementation object.

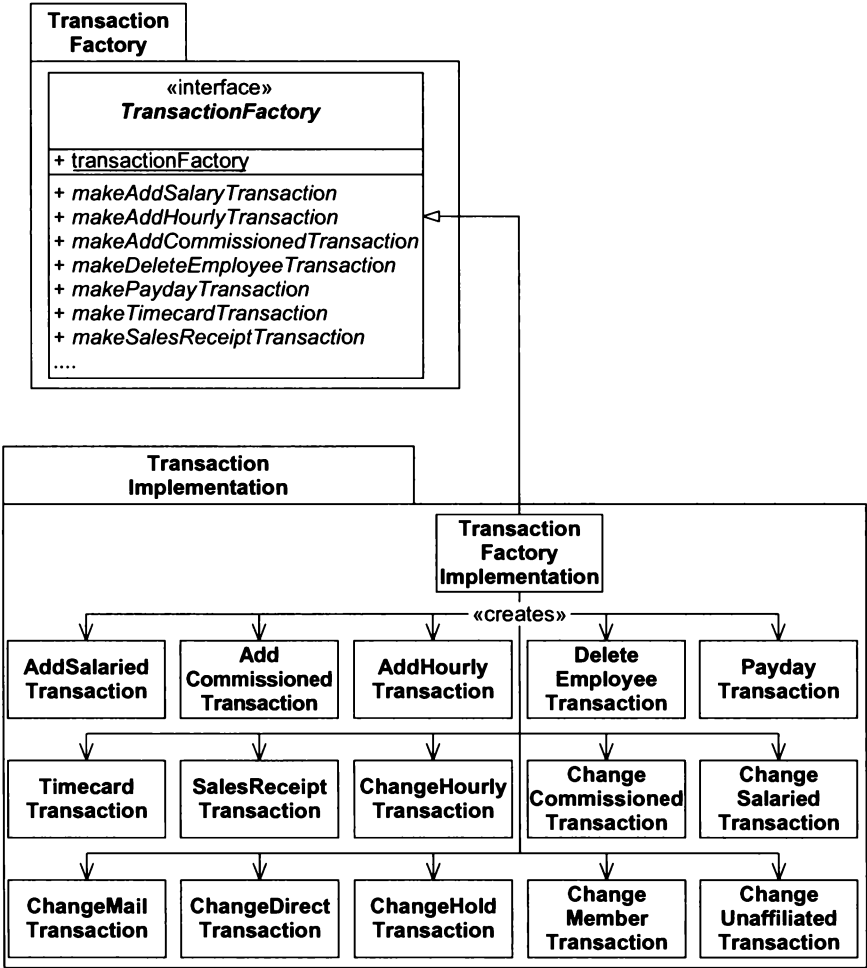


Figure 22-8 Object Factory for Transactions

Initializing the Factories

In order to create objects using the object factories, the static members of the abstract object factories must be initialized to point to the appropriate concrete factory. This must be done before any user attempts to use the factory. The best place to do this is usually the main program, which means that the main program depends on all the factories *and* on all the concrete packages. Thus, each concrete package will have at least one afferent coupling from the main program. This will force the concrete package off the main sequence a bit, but it cannot

be helped.⁶ It means that we must rerelease the main program every time we change any of the concrete packages. Of course we should probably rerelease the main program for each change anyway, since it will need to be tested regardless.

Figures 22-9 and 22-10 show the static and dynamic structure of the main program in relation to the object factories.

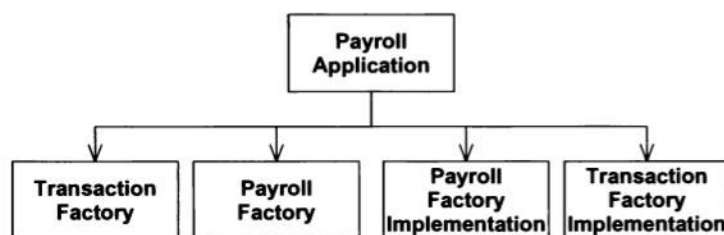


Figure 22-9 Static Structure of Main Program and Object Factories

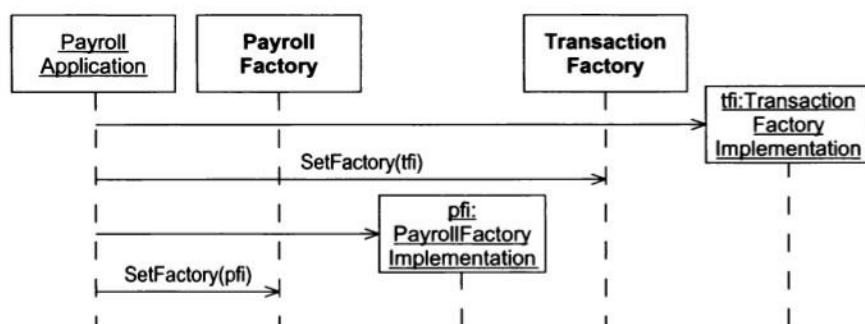


Figure 22-10 Dynamic Structure of Main Program and Object Factories

Rethinking the Cohesion Boundaries

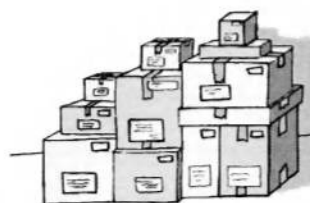
We initially separated *Classifications*, *Methods*, *Schedules*, and *Affiliations* in Figure 22-1. At the time, it seemed like a reasonable partitioning. After all, other users may want to reuse our schedule classes without reusing our affiliation classes. This partitioning was maintained after we split out the transactions into their own packages, creating a dual hierarchy. Perhaps this was too much. The diagram in Figure 22-7 is very tangled.

A tangled package diagram makes the management of releases difficult if it is done by hand. Although package diagrams would work well with an automated project-planning tool, most of us don't have that luxury. Thus, we need to keep our package diagrams as simple as is practical.

In my view, the transaction partitioning is more important than the functional partitioning. Thus, we will merge the transactions into a single *TransactionImplementation* package in Figure 22-11. We will also merge the *Classifications*, *Schedules*, *Methods*, and *Affiliations* packages into a single *PayrollImplementation* package.

The Final Package Structure

Table 22-3 shows the final allocation of classes to class package. Table 22-4 contains the metrics spreadsheet. Figure 22-11 shows the final package structure, which employs object factories to bring the concrete packages near the main sequence



6. As a practical solution, I usually ignore couplings from the main program.

Table 22-3

Packages	Classes in Packages		
AbstractTransactions	AddEmployeeTransaction	ChangeAffiliationTransaction	ChangeEmployee-Transaction
	ChangeClassification-Transaction	ChangeMethodTransaction	
Application	Application		
PayrollApplication	PayrollApplication		
PayrollDatabase	PayrollDatabase		
PayrollDatabaseImple-mentation	PayrollDatabase-Implementation		
PayrollDomain	Affiliation	Employee	PaymentClassification
	PaymentMethod	PaymentSchedule	
PayrollFactory	PayrollFactory		
PayrollImplementation	BiweeklySchedule	CommissionedClassification	DirectMethod
	HoldMethod	HourlyClassification	MailMethod
	MonthlySchedule	PayrollFactory-Implementation	SalariedClassification
	SalesReceipt	ServiceCharge	Timecard
	UnionAffiliation	WeeklySchedule	
TextParser-TransactionSource	TextParserTransactionSource		
Transaction-Application	Transaction	TransactionApplication	TransactionSource
TransactionFactory	TransactionFactory		
Transaction-Implementation	AddCommissionedEmployee	AddHourlyEmployee	AddSalariedEmployee
	ChangeAddressTransaction	ChangeCommissioned-Transaction	ChangeDirectTransaction
	ChangeHoldTransaction	ChangeHourlyTransaction	ChangeMailTransaction
	ChangeMemberTransaction	ChangeNameTransaction	ChangeSalariedTransaction
	ChangeUnaffiliatedTransaction	DeleteEmployee	PaydayTransaction
	SalesReceiptTransaction	ServiceChargeTransaction	TimecardTransaction
	TransactionFactory-Implementation		

Table 22-4

Package Name	N	A	Ca	Ce	R	H	I	A	D	D'
AbstractTransactions	5	5	13	1	0	.20	.07	1	.05	.07
Application	1	1	1	0	0	1	0	1	0	0
PayrollApplication	1	0	0	5	0	1	1	0	0	0
PayrollDatabase	1	1	19	5	0	1	.21	1	.15	.21
PayrollDatabaseImpl...	1	0	0	1	0	1	1	0	0	0
PayrollDomain	5	4	30	0	4	1	0	.80	.14	.20
PayrollFactory	1	1	12	4	0	1	.25	1	.18	.25
PayrollImplementation	14	0	1	5	3	.29	.83	0	.12	.17
TextParserTransactionSource	1	0	1	3	0	1	.75	0	.18	.25
TransactionApplication	3	3	14	1	3	1.33	.07	1	.05	.07
TransactionFactory	1	1	3	1	0	1	.25	1	.18	.25
TransactionImplementation	19	0	1	14	0	.05	.93	0	.05	.07

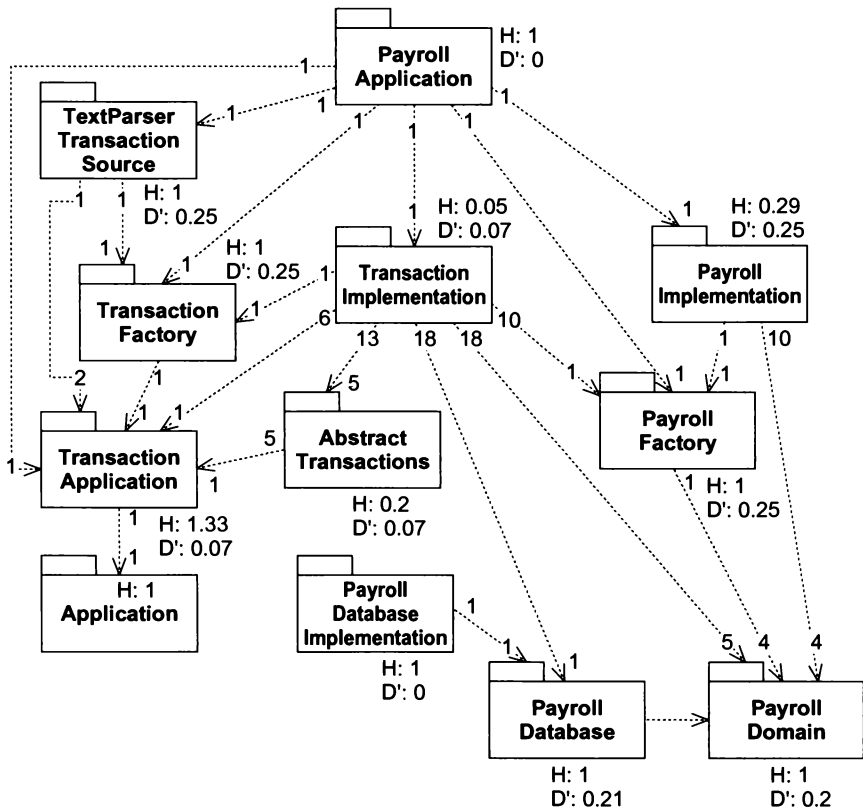


Figure 22-11 Final Payroll Package Structure

The metrics on this diagram are heartening. The relational cohesions are all very high (thanks in part to the relationships of the concrete factories to the objects that they create), and there are no significant deviations from the main sequence. Thus, the couplings between our packages are appropriate to a sane development environment. Our abstract packages are closed, reusable, and heavily depended on, while having few dependencies of their own. Our concrete packages are segregated on the basis of reuse, are heavily dependent on the abstract packages, and are not heavily depended on themselves.

Conclusion

The need to manage package structures is a function of the size of the program and the size of the development team. Even small teams need to partition the source code so that they can stay out of each other's way. Large programs can become opaque masses of source files without some kind of partitioning structure.

Bibliography

1. Benjamin/Cummings. *Object-Oriented Analysis and Design with Applications*, 2d ed., 1994.
2. DeMarco, Tom. *Controlling Software Projects*. Yourdon Press, 1982.

SECTION 5

The Weather Station Case Study

The following chapters contain an in-depth case study of a simple weather monitoring system. Although this case study is fictitious, it has nevertheless been constructed with a high degree of realism. We will encounter the problems of time pressure, legacy code, poor and mutating specifications, new untried technologies, etc. Our goal is to demonstrate how the principles, patterns, and practices that we have learned are used in the real world of software engineering.

As before, we will be encountering several useful design patterns while exploring the development of the weather station. The chapters leading up to the case study will describe those patterns.

COMPOSITE



The COMPOSITE pattern is a very simple pattern that has significant implications. The fundamental structure of the COMPOSITE pattern is shown in Figure 23-1. Here we see a hierarchy based on shapes. The Shape base class has two derivative shapes named Circle and Square. The third derivative is the composite. CompositeShape keeps a list of many Shape instances. When draw() is called on CompositeShape, it delegates that method to all the Shape instances in the list.

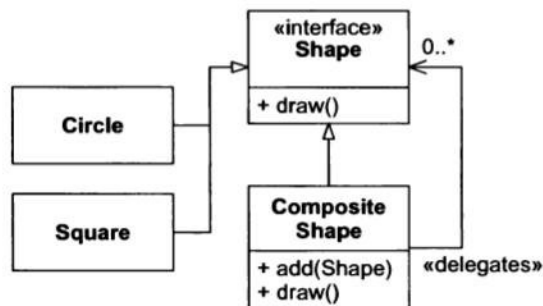


Figure 23-1 Composite Pattern

Thus, an instance of `CompositeShape` appears to the system to be a single `Shape`. It can be passed to any function or object that takes a `Shape`, and it will behave like a `Shape`. However, it is really a proxy¹ for a group of `Shape` instances.

Listing 23-1 and Listing 23-2 show one possible implementation of `CompositeShape`.

Listing 23-1

Shape.java

```
public interface Shape
{
    public void draw();
}
```

Listing 23-2

CompositeShape.java

```
import java.util.Vector;

public class CompositeShape implements Shape
{
    private Vector itsShapes = new Vector();
    public void add(Shape s)
    {
        itsShapes.add(s);
    }

    public void draw()
    {
        for (int i = 0; i < itsShapes.size(); i++)
        {
            Shape shape = (Shape) itsShapes.elementAt(i);
            shape.draw();
        }
    }
}
```

Example: Composite Commands

Consider the discussion of `Sensors` and `Command` objects we had back on page 152. Figure 13-3 showed a `Sensor` class using a `Command` class. When the `Sensor` detected its stimulus, it called `do()` on the `Command`.

What I failed to mention in that discussion was that there were often cases when a `Sensor` had to execute more than one `Command`. For example, when the paper reached a certain point in the paper path, it would trip an optical sensor. That sensor then stopped a motor, started another, and engaged a particular clutch.

At first we took this to mean that every `Sensor` class would have to maintain a list of `Command` objects. (See Figure 23-2.) However, we soon recognized that whenever a `Sensor` needed to execute more than one `Command`, it always treated those `Command` objects identically. That is, it just iterated over the list and called `do()` on each `Command`. This was ideal for the COMPOSITE pattern.

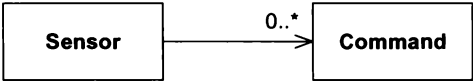


Figure 23-2 Sensor containing many Commands

1. Notice the similarity in structure to the PROXY pattern.

So we left the `Sensor` class alone and created a `CompositeCommand` as shown in Figure 23-3.

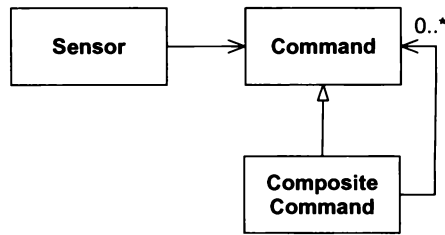


Figure 23-3 Composite Command

This meant that we didn't have to change the `Sensor` or the `Command`. We were able to add the plurality of `Commands` to a `Sensor` without changing either. This is an application of the OCP.

Multiplicity or Not Multiplicity

This leads to an interesting issue. We were able to make our `Sensors` behave as though they contained many `Commands`, without having to modify the `Sensors`. There must be many other situations like this in normal software design. There must be times when you could use `COMPOSITE` rather than building a list or vector of objects.

Let me say this a different way. The association between `Sensor` and `Command` is one-to-one. We were tempted to change that association to one to many. But instead we found a way to get one-to-many behavior, without a one-to-many relationship. A one-to-one relationship is much easier to understand, code, and maintain than a one-to-many relationship is; so this was clearly the right design trade-off. How many of the one-to-many relationships in your current project could be one-to-one if you used `COMPOSITE`?

Of course, not all one-to-many relationships can be reverted to one-to-one by using `COMPOSITE`. Only those in which every object in the list is treated identically are candidates. For example, if you maintained a list of employees and searched through that list for employees whose pay date is today, you probably shouldn't use the `COMPOSITE` pattern because you wouldn't be treating all the employees identically.

Still, there are quite a few one-to-many relationships that qualify for conversion to `COMPOSITE`. And the advantages are significant. Instead of duplicating the list management and iteration code in each of the clients, that code appears only once in the composite class.

OBSERVER—Backing into a Pattern



This chapter serves a special purpose. In it, I will describe the OBSERVER¹ pattern, but that is a minor objective. The primary objective of this chapter is to give you a demonstration of how your design and code can evolve to use a pattern.

In the preceding chapters, we have made use of many patterns. Often we presented them as a *fait accompli*, without showing how the code evolved to use the pattern. This might give you the idea that patterns are simply something you insert into your code and designs in completed form. This is not what I advise. Rather, I prefer to evolve the code I am working on in the direction of its needs. As I refactor it to resolve issues of coupling, simplicity, and expressiveness, I may find that the code has come close to a particular pattern. When that happens, I change the names of the classes and variables to use the name of the pattern, and change the structure of the code to use the pattern in a more regular form. Thus the code *backs into the pattern*.

This chapter sets up a simple problem and then shows how the design and code evolve to solve that problem. The result of the evolution winds up being the OBSERVER pattern. At each stage of the evolution, I will describe the issues I'm trying to resolve, and then show the steps that resolve them.

The Digital Clock

We have a clock object. This object catches millisecond interrupts (known as ticks) from the operating system and turns them into the time of day. This object knows how to calculate seconds from milliseconds, minutes from seconds, hours from minutes, days from hours, etc. It knows how many days are in a month, and how many months are in a year. It knows all about leap years, when to have them, and when not. It knows about time. (See Figure 24-1.)

1. [GOF95], p. 293.

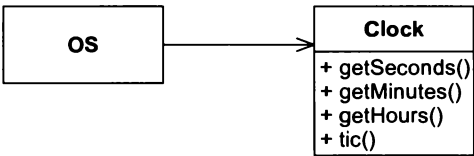


Figure 24-1 Clock

We’d like to create a digital clock that sits on our desktop and continuously displays the time of day. What is the simplest way to accomplish this? We could write the following code:

```
public void DisplayTime
{
    while(1)
    {
        int sec = clock.getSeconds();
        int min = clock.getMinutes();
        int hour = clock.getHours();
        showTime(hour,min,sec);
    }
}
```

Clearly this is suboptimal. It consumes all available CPU cycles to repeatedly display the time. Most of those displays will be wasted because the time will not have changed. It may be that this solution would be adequate in a digital watch or a digital wall clock since conserving CPU cycles is not very important in those systems. However, we don’t want this CPU hog running on our desktop.

The fundamental problem is how to efficiently get data from the `Clock` to the `DigitalClock`. I’m going to assume that the `Clock` object and the `DigitalClock` object both exist. My interest is in how to connect them. I can test that connection simply by making sure that the data I get from the `Clock` are the same data I send to the `DigitalClock`.

A simple way to write this test is to create one interface that pretends to be the `Clock` and another that pretends to be the `DigitalClock`. Then I can write special test objects that implement those interfaces and verify that the connection between them works as expected. (See Figure 24-2.)

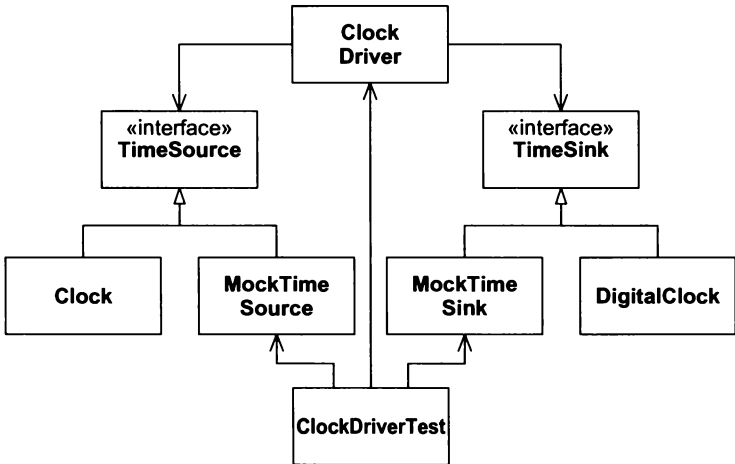


Figure 24-2 Testing the Digital Clock

The `ClockDriverTest` object will connect the `ClockDriver` to the two mock objects through the `TimeSource` and `TimeSink` interfaces. It will then check each of the mock objects to ensure that the `ClockDriver` manages to move the time from the source to the sink. If necessary, the `ClockDriverTest` will also ensure that efficiency is being conserved.

I think it's interesting that we have added interfaces to the design simply as a result of considering how to test it. In order to test a module, you have to be able to isolate it from the other modules in the system, just as we have isolated the `ClockDriver` from the `Clock` and `DigitalClock`. Considering tests first helps us to minimize the coupling in our designs.

OK, how does the `ClockDriver` work? Clearly, in order to be efficient, the `ClockDriver` must detect when the time in the `TimeSource` object has changed. Then, and only then, should it move the time to the `TimeSink` object. How can the `ClockDriver` know when the time has changed? It could poll the `TimeSource`, but that simply recreates the CPU hog problem.

The simplest way for the `ClockDriver` to know when the time has changed is for the `Clock` object to tell it. We could pass the `ClockDriver` to the `Clock` through the `TimeSource` interface and then, when the time changes, the `Clock` can update the `ClockDriver`. The `ClockDriver` will, in turn, set the time on the `ClockSink`. (See Figure 24-3.)

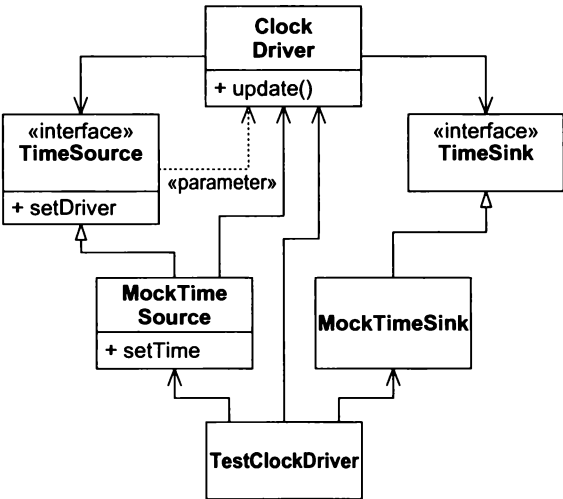


Figure 24-3 Getting the `TimeSource` to update the `ClockDriver`

Notice the dependency from the `TimeSource` to the `ClockDriver`. It is there because the argument to the `setDriver` method is a `ClockDriver`. I'm not very happy with this, since it implies that `TimeSource` objects must use `ClockDriver` objects in every case. However, I'll defer doing anything about the dependency until I get this program working.

Listing 24-1 shows the test case for the `ClockDriver`. Notice that it creates a `ClockDriver` and binds a `MockTimeSource` and a `MockTimeSink` to it. Then it sets the time in the source and expects the time to magically arrive at the sink. The rest of the code is shown in Listings 24-2 through 24-6.

Listing 24-1

`ClockDriverTest.java`

```
import junit.framework.*;

public class ClockDriverTest extends TestCase
{
    public ClockDriverTest(String name)
```

```

{
    super(name);
}

public void testTimeChange()
{
    MockTimeSource source = new MockTimeSource();
    MockTimeSink sink = new MockTimeSink();
    ClockDriver driver = new ClockDriver(source, sink);
    source.setTime(3,4,5);
    assertEquals(3, sink.getHours());
    assertEquals(4, sink.getMinutes());
    assertEquals(5, sink.getSeconds());

    source.setTime(7,8,9);
    assertEquals(7, sink.getHours());
    assertEquals(8, sink.getMinutes());
    assertEquals(9, sink.getSeconds());
}
}

```

Listing 24-2**TimeSource.java**

```

public interface TimeSource
{
    public void setDriver(ClockDriver driver);
}

```

Listing 24-3**TimeSink.java**

```

public interface TimeSink
{
    public void setTime(int hours, int minutes, int seconds);
}

```

Listing 24-4**ClockDriver.java**

```

public class ClockDriver
{
    private TimeSink itsSink;

    public ClockDriver(TimeSource source, TimeSink sink)
    {
        source.setDriver(this);
        itsSink = sink;
    }

    public void update(int hours, int minutes, int seconds)
    {
        itsSink.setTime(hours, minutes, seconds);
    }
}

```

Listing 24-5**MockTimeSource.java**

```
public class MockTimeSource implements TimeSource
{
    private ClockDriver itsDriver;

    public void setTime(int hours, int minutes, int seconds)
    {
        itsDriver.update(hours, minutes, seconds);
    }

    public void setDriver(ClockDriver driver)
    {
        itsDriver = driver;
    }
}
```

Listing 24-6**MockTimeSink.java**

```
public class MockTimeSink implements TimeSink
{
    private int itsHours;
    private int itsMinutes;
    private int itsSeconds;

    public int getSeconds()
    {
        return itsSeconds;
    }

    public int getMinutes()
    {
        return itsMinutes;
    }

    public int getHours()
    {
        return itsHours;
    }

    public void setTime(int hours, int minutes, int seconds)
    {
        itsHours = hours;
        itsMinutes = minutes;
        itsSeconds = seconds;
    }
}
```

OK, now that it works, I can think about cleaning it up. I don't like the dependency from `TimeSource` to `ClockDriver` because I want the `TimeSource` interface to be usable by anybody, not just `ClockDriver` objects. We can fix this by creating an interface that `TimeSource` can use and that `ClockDriver` can implement. We'll call this interface `ClockObserver`. See Listings 24-7 through 24-10. The code in **bold** has changed.

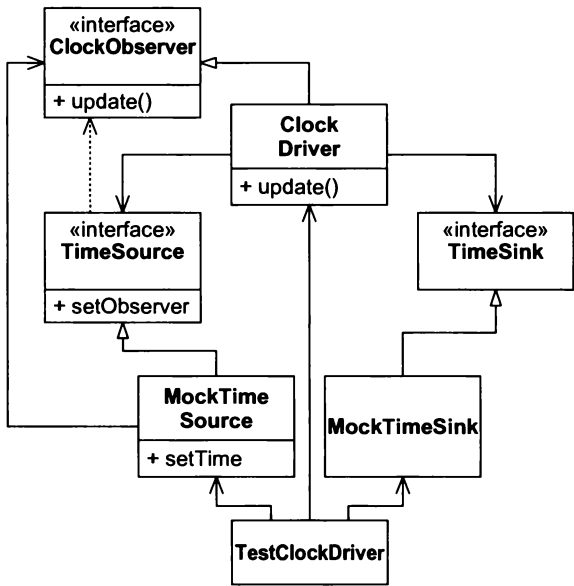


Figure 24-4 Breaking the dependency of TimeSource upon ClockDriver

Listing 24-7

ClockObserver.java

```
public interface ClockObserver
{
    public void update(int hours, int minutes, int seconds);
}
```

Listing 24-8

ClockDriver.java

```
public class ClockDriver implements ClockObserver
{
    private TimeSink itsSink;

    public ClockDriver(TimeSource source, TimeSink sink)
    {
        source.setObserver(this);
        itsSink = sink;
    }

    public void update(int hours, int minutes, int seconds)
    {
        itsSink.setTime(hours, minutes, seconds);
    }
}
```

Listing 24-9

TimeSource.java

```
public interface TimeSource
{
    public void setObserver(ClockObserver observer);
}
```

Listing 24-10

MockTimeSource.java

```
public class MockTimeSource implements TimeSource
{
    private ClockObserver itsObserver;

    public void setTime(int hours, int minutes, int seconds)
    {
        itsObserver.update(hours, minutes, seconds);
    }

    public void setObserver(ClockObserver observer)
    {
        itsObserver = observer;
    }
}
```

This is better. Now anybody can make use of `TimeSource`. All they have to do is implement `ClockObserver` and call `setObserver`, passing themselves in as the argument.

I'd like to be able to have more than one `TimeSink` getting the time. One might implement a digital clock. Another might be used to supply the time to a reminder service. Still another might start my nightly backup. In short, I'd like a single `TimeSource` to be able to supply the time to multiple `TimeSink` objects.

So I'll change the constructor of the `ClockDriver` to take just the `TimeSource` and then add a method named `addTimeSink` that allows you to add `TimeSink` instances any time you want.

The thing I don't like about this is that I now have two indirections. I have to tell the `TimeSource` who the `ClockObserver` is by calling `setObserver`, and I also have to tell the `ClockDriver` who the `TimeSink` instances are. Is this double indirection really necessary?

Looking at `ClockObserver` and `TimeSink`, I see that they both have the `setTime` method. It looks like `TimeSink` could implement `ClockObserver`. If I did this, then my test program could create a `MockTimeSink` and call `setObserver` on the `TimeSource`. I could get rid of the `ClockDriver` (and `TimeSink`) altogether! Listing 24-11 shows the changes to `ClockDriverTest`.

Listing 24-11

ClockDriverTest.java

```
import junit.framework.*;

public class ClockDriverTest extends TestCase
{
    public ClockDriverTest(String name)
    {
        super(name);
    }

    public void testTimeChange()
    {
        MockTimeSource source = new MockTimeSource();
        MockTimeSink sink = new MockTimeSink();
        source.setObserver(sink);

        source.setTime(3,4,5);
        assertEquals(3, sink.getHours());
    }
}
```

```

    assertEquals(4, sink.getMinutes());
    assertEquals(5, sink.getSeconds());

    source.setTime(7,8,9);
    assertEquals(7, sink.getHours());
    assertEquals(8, sink.getMinutes());
    assertEquals(9, sink.getSeconds());
}
}

```

This means that `MockTimeSink` should implement `ClockObserver` rather than `TimeSink`. See Listing 24-12. These changes work fine. Why did I think I needed a `ClockDriver` in the first place? Figure 24-5 shows the UML.

Listing 24-12

MockTimeSink.java

```

public class MockTimeSink implements ClockObserver
{
    private int itsHours;
    private int itsMinutes;
    private int itsSeconds;

    public int getSeconds()
    {
        return itsSeconds;
    }

    public int getMinutes()
    {
        return itsMinutes;
    }

    public int getHours()
    {
        return itsHours;
    }

    public void update(int hours, int minutes, int seconds)
    {
        itsHours = hours;
        itsMinutes = minutes;
        itsSeconds = seconds;
    }
}

```

Clearly this is much simpler.

OK, now we can handle multiple `TimeSink` objects by changing the `setObserver` function to register-Observer and by making sure that all the registered `ClockObserver` instances are held in a list and updated appropriately. This requires another change to the test program. Listing 24-13 shows the changes. I also did a little refactoring of the test program to make it smaller and easier to read.

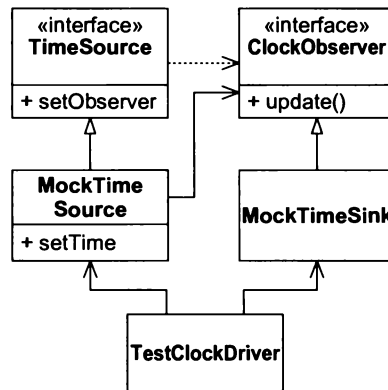


Figure 24-5 Removing ClockDriver and TimeSink

Listing 24-13

ClockDriverTest.java

```

import junit.framework.*;

public class ClockDriverTest extends TestCase
{
    private MockTimeSource source;
    private MockTimeSink sink;

    public ClockDriverTest(String name)
    {
        super(name);
    }

    public void setUp()
    {
        source = new MockTimeSource();
        sink = new MockTimeSink();
        source.registerObserver(sink);
    }

    private void assertSinkEquals(
        MockTimeSink sink, int hours, int minutes, int seconds)
    {
        assertEquals(hours, sink.getHours());
        assertEquals(minutes, sink.getMinutes());
        assertEquals(seconds, sink.getSeconds());
    }

    public void testTimeChange()
    {
        source.setTime(3,4,5);
        assertSinkEquals(sink, 3,4,5);

        source.setTime(7,8,9);
        assertSinkEquals(sink, 7,8,9);
    }
}

```

```

public void testMultipleSinks()
{
    MockTimeSink sink2 = new MockTimeSink();
    source.registerObserver(sink2);

    source.setTime(12,13,14);
    assertSinkEquals(sink, 12,13,14);
    assertSinkEquals(sink2, 12,13,14);
}
}

```

The change needed to make this work is pretty simple. We change `MockTimeSource` to hold all registered observers in a `Vector`. Then, when the time changes, we iterate through the `Vector` and call `update` on all the registered `ClockObservers`. Listings 24-14 and 24-15 show the changes. Figure 24-6 shows the corresponding UML.

Listing 24-14

TimeSource.java

```

public interface TimeSource
{
    public void registerObserver(ClockObserver observer);
}

```

Listing 24-15

MockTimeSource.java

```

import java.util.*;

public class MockTimeSource implements TimeSource
{
    private Vector itsObservers = new Vector();

    public void setTime(int hours, int minutes, int seconds)
    {
        Iterator i = itsObservers.iterator();
        while (i.hasNext())
        {
            ClockObserver observer = (ClockObserver) i.next();
            observer.update(hours, minutes, seconds);
        }
    }

    public void registerObserver(ClockObserver observer)
    {
        itsObservers.add(observer);
    }
}

```

This is pretty nice, but I don't like the fact that the `MockTimeSource` has to deal with the registration and update. It implies that the `Clock` and every other derivative of `TimeSource` will have to duplicate that registration and update code. I don't think `Clock` should have to deal with registration and update. I also don't like the idea of duplicate code. So I'd like to move all that stuff into the `TimeSource`. Of course, this means that `TimeSource`

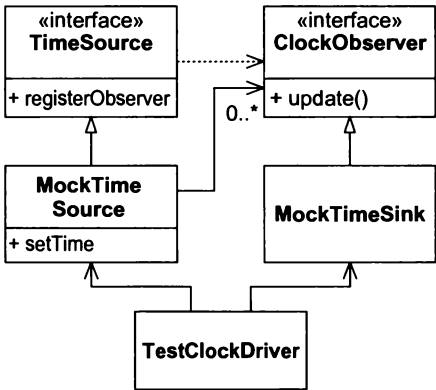


Figure 24-6 Handling multiple TimeSink objects

will have to change from an interface to a class. It also means that MockTimeSource will shrink to near nothing. Listings 24-16 and 24-17 and Figure 24-7 show the changes.

Listing 24-16

TimeSource.java

```
import java.util.*;

public class TimeSource
{
    private Vector itsObservers = new Vector();

    protected void notify(int hours, int minutes, int seconds)
    {
        Iterator i = itsObservers.iterator();
        while (i.hasNext())
        {
            ClockObserver observer = (ClockObserver) i.next();
            observer.update(hours, minutes, seconds);
        }
    }

    public void registerObserver(ClockObserver observer)
    {
        itsObservers.add(observer);
    }
}
```

Listing 24-17

MockTimeSource.java

```
public class MockTimeSource extends TimeSource
{
    public void setTime(int hours, int minutes, int seconds)
    {
        notify(hours, minutes, seconds);
    }
}
```

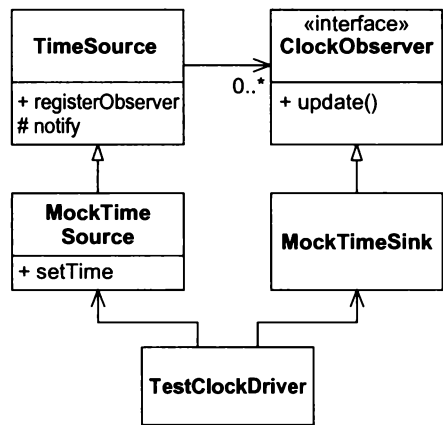


Figure 24-7 Moving registration and update into TimeSource

This is pretty cool. Now, anybody can derive from `TimeSource`. All they have to do to get the observers updated is to call `notify`. But there is still something I don't like about it. `MockTimeSource` inherits directly from `TimeSource`. This means that `Clock` must also derive from `TimeSource`. Why should `Clock` have to depend upon registration and update? `Clock` is just a class that knows about time. Making it depend upon `TimeSource` seems necessary and undesirable.

I know how I'd solve this in C++. I'd create a subclass of both `TimeSource` and `Clock` called `ObservableClock`. I'd override `tic` and `setTime` in `ObservableClock` to call `tic` or `setTime` in `Clock` and then call `notify` in `TimeSource`. See Listing 24-18 and Figure 24-8.

Listing 24-18

ObservableClock.cc (C++)

```
class ObservableClock : public Clock, public TimeSource
{
public:
    virtual void tic()
    {
        Clock::tic();
        TimeSource::notify(getHours(), getMinutes(), getSeconds());
    }

    virtual void setTime(int hours, int minutes, int seconds)
    {
        Clock::setTime(hours, minutes, seconds);
        TimeSource::notify(hours, minutes, seconds);
    }
};
```

Unfortunately, we don't have this option in Java because the language can't deal with multiple inheritance of classes. So, in Java we either have to leave things as they are or use a delegation hack. The delegation hack is shown in Listings 24-19 through 24-21 and Figure 24-9.

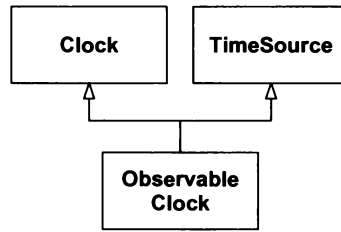


Figure 24-8 Using multiple inheritance in C++ to separate Clock from TimeSource

Listing 24-19

TimeSource.java

```
public interface TimeSource
{
    public void registerObserver(ClockObserver observer);
}
```

Listing 24-20

TimeSourceImplementation.java

```
import java.util.*;

public class TimeSourceImplementation
{
    private Vector itsObservers = new Vector();

    public void notify(int hours, int minutes, int seconds)
    {
        Iterator i = itsObservers.iterator();
        while (i.hasNext())
        {
            ClockObserver observer = (ClockObserver) i.next();
            observer.update(hours, minutes, seconds);
        }
    }

    public void registerObserver(ClockObserver observer)
    {
        itsObservers.add(observer);
    }
}
```

Listing 24-21

MockTimeSource.java

```
public class MockTimeSource implements TimeSource
{
    TimeSourceImplementation tsImp =
        new TimeSourceImplementation();

    public void registerObserver(ClockObserver observer)
    {
        tsImp.registerObserver(observer);
    }
}
```

```
public void setTime(int hours, int minutes, int seconds)
{
    tsImp.notify(hours, minutes, seconds);
}
}
```

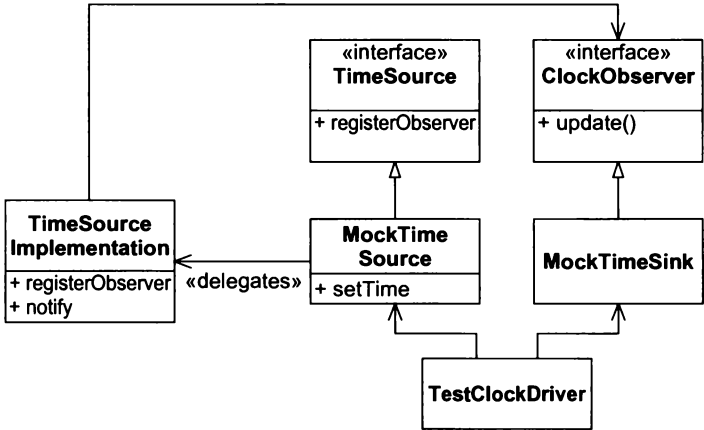


Figure 24-9 Observer delegation hack in Java

Notice that the `MockTimeSource` class implements `TimeSource` and contains a reference to an instance of `TimeSourceImplementation`. Notice also that all calls to the `registerObserver` method of `MockTimeSource` are delegated to that `TimeSourceImplementation` object. So too, `MockTimeSource.setTime` invokes `notify` on the `TimeSourceImplementation` instance.

This is ugly, but it has the advantage that `MockTimeSource` does not extend a class. This means that if we were to create `ObservableClock`, it could extend `Clock`, implement `TimeSource`, and delegate to `TimeSourceImplementation`. (See Figure 24-10.) This solves the problem of `Clock` depending upon the registration and update stuff, but at a nontrivial price.

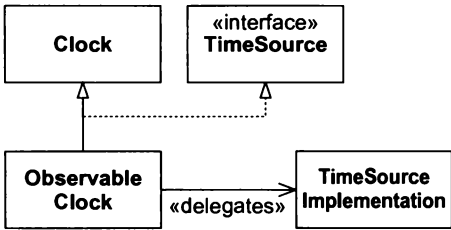


Figure 24-10 The delegation hack for `ObservableClock`

So, let’s go back to the way things were in Figure 24-7, before we went down this rathole. We’ll simply live with the fact that `Clock` has to depend upon all the registration and update stuff.

`TimeSource` is a stupid name for what the class does. It started out good, back in the days when we had a `ClockDriver`. But things have changed an awful lot since then. We should change the name to something that suggests registration and update. The `OBSERVER` pattern calls this class `Subject`. Ours seems to be specific to time, so we could call it `TimeSubject`, but that’s not a very intuitive name. We could use the old Java moniker `Observable`, but that doesn’t ring my chimes either. `TimeObservable`?—No.

Perhaps it is the specificity of the “push-model” observer that is the problem.² If we change to a “pull model” we could make the class generic. Then we could change the name of TimeSource to Subject, and everybody familiar with the OBSERVER pattern would know what it meant.

This is not a bad option. Rather than pass the time in the notify and update methods, we can have the TimeSink ask the MockTimeSource for the time. We don’t want the MockTimeSink to know about the MockTimeSource, so we’ll create an interface that the MockTimeSink can use to get the time. The MockTimeSource (and the Clock) will implement this interface. We’ll call this interface—er—TimeSource.

The final state of the code and UML are in Figure 24-11 and Listings 24-22 through 24-27.

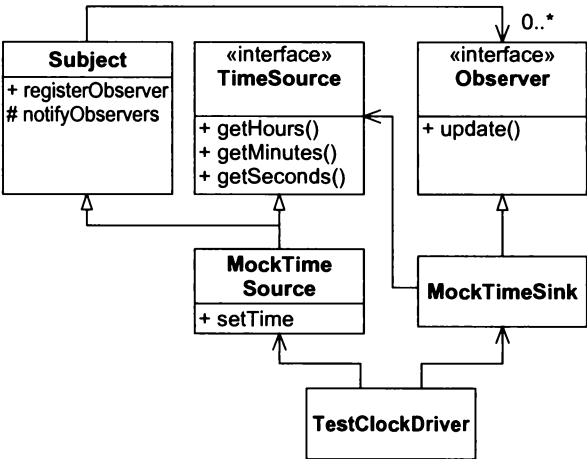


Figure 24-11 Final version of the Observer applied to MockTimeSource and MockTimeSink

Listing 24-22

ObserverTest.java

```
import junit.framework.*;

public class ObserverTest extends TestCase
{
    private MockTimeSource source;
    private MockTimeSink sink;

    public ObserverTest(String name)
    {
        super(name);
    }

    public void setUp()
    {
        source = new MockTimeSource();
        sink = new MockTimeSink(source);
        source.registerObserver(sink);
    }

    private void assertSinkEquals(
```

2. “Push-model” observers push data from the subject to the observer by passing it in the notify and update methods. “Pull-model” observers pass nothing in the notify and update methods, and they depend upon the observing object to query the observed object upon receiving an update. See [GOF95].

```

private void assertSinkEquals(
    MockTimeSink sink, int hours, int minutes, int seconds)
{
    assertEquals(hours, sink.getHours());
    assertEquals(minutes, sink.getMinutes());
    assertEquals(seconds, sink.getSeconds());
}

public void testTimeChange()
{
    source.setTime(3,4,5);
    assertSinkEquals(sink, 3,4,5);

    source.setTime(7,8,9);
    assertSinkEquals(sink, 7,8,9);
}

public void testMultipleSinks()
{
    MockTimeSink sink2 = new MockTimeSink(source);
    source.registerObserver(sink2);

    source.setTime(12,13,14);
    assertSinkEquals(sink, 12,13,14);
    assertSinkEquals(sink2, 12,13,14);
}
}

```

Listing 24-23

Observer.java

```

public interface Observer
{
    public void update();
}

```

Listing 24-24

Subject.java

```

import java.util.*;

public class Subject
{
    private Vector itsObservers = new Vector();

    protected void notifyObservers()
    {
        Iterator i = itsObservers.iterator();
        while (i.hasNext())
        {
            Observer observer = (Observer) i.next();
            observer.update();
        }
    }
}

```

```
public void registerObserver(Observer observer)
{
    itsObservers.add(observer);
}
}
```

Listing 24-25**TimeSource.java**

```
public interface TimeSource
{
    public int getHours();
    public int getMinutes();
    public int getSeconds();
}
```

Listing 24-26**MockTimeSource.java**

```
public class MockTimeSource extends Subject
                               implements TimeSource
{
    private int itsHours;
    private int itsMinutes;
    private int itsSeconds;

    public void setTime(int hours, int minutes, int seconds)
    {
        itsHours = hours;
        itsMinutes = minutes;
        itsSeconds = seconds;
        notifyObservers();
    }

    public int getHours()
    {
        return itsHours;
    }

    public int getMinutes()
    {
        return itsMinutes;
    }

    public int getSeconds()
    {
        return itsSeconds;
    }
}
```

Listing 24-27**MockTimeSink.java**

```
public class MockTimeSink implements Observer
{
    private int itsHours;
```

```

private int itsMinutes;
private int itsSeconds;
private TimeSource itsSource;

public MockTimeSink(TimeSource source)
{
    itsSource = source;
}

public int getSeconds()
{
    return itsSeconds;
}

public int getMinutes()
{
    return itsMinutes;
}

public int getHours()
{
    return itsHours;
}

public void update()
{
    itsHours = itsSource.getHours();
    itsMinutes = itsSource.getMinutes();
    itsSeconds = itsSource.getSeconds();
}
}

```

Conclusion

So, we made it. We started with a design problem and, through reasonable evolution, wound up at the canonical OBSERVER pattern. You might complain that since I knew that I wanted to arrive at the OBSERVER I simply arranged it so that I would. I won't deny it. But that's not really the issue.

If you are familiar with design patterns, then when faced with a design problem, a pattern will very likely pop into your mind. The question then is whether or not to implement that pattern directly, or to continue to evolve the code through a series of small steps. This chapter showed what the second option is like. Rather than simply leaping to the conclusion that the OBSERVER pattern was the best choice for the problem at hand, I continued to resolve the issues one by one. Eventually it was pretty clear that the code was heading in the direction of OBSERVER, so I changed the names and put the code into canonical form.

At any point during that evolution, I could have found that my problem was solved and stopped evolving. Or, I might have found that I could solve the problem by changing course and going in a different direction.

The Use of Diagrams in this Chapter

Some of the diagrams I drew for your benefit. I thought it would be easier for you to follow what I was doing by showing you an overview in a diagram. Had I not been trying to expose and expound, I would not have created them. However, *a few* of the diagrams were created for *my* benefit. There were times when I just needed to stare at the structure that I had created so I could see where to go next.

Had I not been writing a book, I would have drawn these diagrams by hand on a scrap of paper or a white-board. I would not have taken the time to use a drawing tool. There are no circumstances that I know of where using a drawing tool is faster than a napkin.

Having used the diagrams to help me evolve the code, I would not have kept the diagrams. In every case, the ones I drew for myself were intermediate steps.

Is there value in keeping diagrams at this level of detail? Clearly, if you are trying to expose your reasoning, as I am doing in this book, they come in pretty handy. But usually we are not trying to document the evolutionary path of a few hours of coding. Usually, these diagrams are transient and are better thrown away. At *this* level of detail, the code is generally good enough to act as its own documentation. At higher levels, that is not always true.

The OBSERVER Pattern

OK, so now that we’ve been through the example and evolved our code to the OBSERVER pattern, it might be interesting to study just what the OBSERVER pattern is. The canonical form of OBSERVER is shown in Figure 24-12. In this example, Clock is being observed by DigitalClock. DigitalClock registers with the Subject interface of Clock. Clock calls the notify method of Subject whenever the time changes for any reason. The notify method of Subject invokes the update method of each registered Observer. Thus, DigitalClock will receive an update message whenever the time changes. It uses that opportunity to ask Clock for the time and then display it.

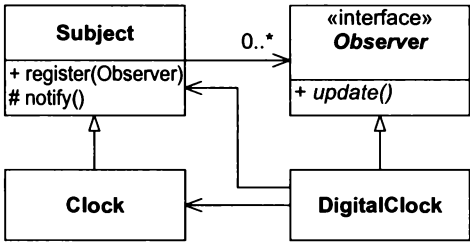


Figure 24-12 Canonical Pull-Model OBSERVER

OBSERVER is one of those patterns that, once you understand it, you see uses for it everywhere. The indirection is very cool. You can register observers with all kinds of objects rather than writing those objects to explicitly call you. While this indirection is a useful way to manage dependencies, it can easily be taken to extremes. Over-use of OBSERVER tends to make systems very difficult to understand and trace.

Push-me-pull-u. There are two primary models of the OBSERVER pattern. Figure 24-13 shows the *pull model* OBSERVER. It gets its name from the fact that the DigitalClock must pull the time information from the Clock object after receiving the update message.

The advantage of the pull model is its simplicity of implementation and the fact that the Subject and Observer classes can be standard reusable elements in a library. However, imagine that you are observing an employee record with a thousand fields and that you have just received an update message. Which of the thousand fields changed?

When update is called on the ClockObserver, the response is obvious. The ClockObserver needs to pull the time from the Clock and display it. But when update is called on the EmployeeObserver, the response is not so obvious. We don’t know what happened. We don’t know what to do. Perhaps the employee’s name changed, or maybe it was his salary. Maybe he got a new boss. Or maybe his bank account changed. We need help.

This help can be given in the push-model form of the OBSERVER pattern. The structure of the push-model observer is shown in Figure 24-13. Notice that the notify and update methods both take an argument. The argument is a hint, passed from Employee to SalaryObserver through the notify and update methods. That hint tells SalaryObserver the kind of change the Employee record experienced.

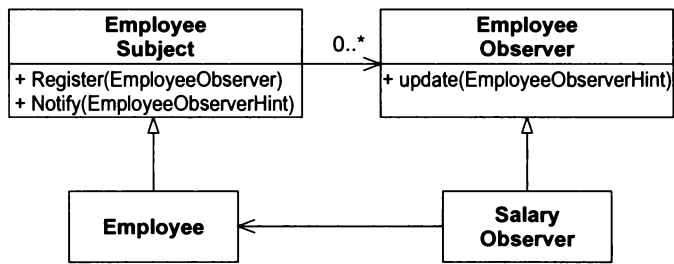


Figure 24-13 Push-Model OBSERVER

The `EmployeeObserverHint` argument of `notify` and `update` might be an enumeration of some kind, a string, or a more complex data structure that contains the old and new values of some field. Whatever it is, its value is being pushed toward the observer.

Choosing between the two different OBSERVER models is simply a matter of the complexity of the observed object. If the observed object is complex, and the observer needs a hint, then the push model is appropriate. If the observed object is simple, then a pull model will do fine.

How OBSERVER Manages the Principles of OOD

The principle that most drives the OBSERVER pattern is the Open–Closed Principle (OCP). The motivation for using the pattern is so that you can add new observing objects without changing the observed object. Thus the observed object stays closed.

Looking back on Figure 24-12, it should be clear that `Clock` is substitutable for `Subject` and that `DigitalClock` is substitutable for `Observer`. Thus, the Liskov Substitution Principle (LSP) is applied.

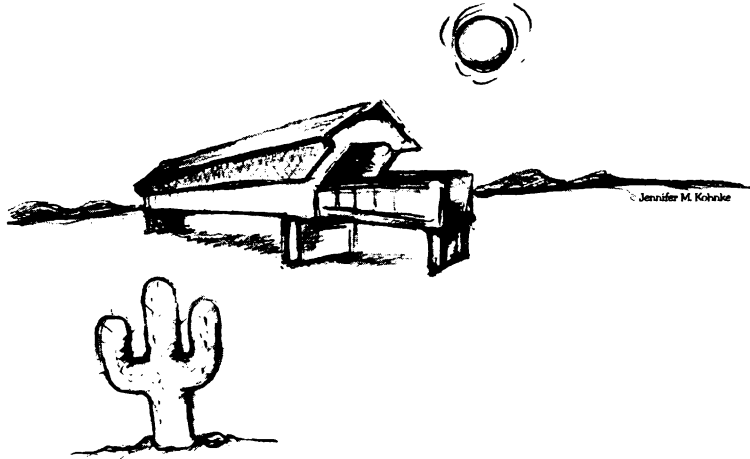
`Observer` is an abstract class, and the concrete `DigitalClock` depends upon it. The concrete methods of `Subject` also depend upon it. Hence, the Dependency-Inversion Principle (DIP) is applied in this case. You might think that since `Subject` has no abstract methods, the dependency between `Clock` and `Subject` violates the DIP. However, `Subject` is a class that ought never to be instantiated. It only makes sense in the context of a derived class. Thus, `Subject` is *logically* abstract, even though it has no abstract methods. We can enforce the abstractness of `Subject` by giving it a pure virtual destructor in C++ or by making its constructors protected.

There are hints of the Interface-Segregation Principle (ISP) in Figure 24-11. The `Subject` and `TimeSource` classes segregate the clients of the `MockTimeSource`, providing specialized interfaces for each of those clients.

Bibliography

- 1. Gamma, et al. *Design Patterns*. Addison–Wesley, 1995.
- 2. Martin, Robert C., et al. *Pattern Languages of Program Design 3*, Addison–Wesley, 1998.

ABSTRACT SERVER, ADAPTER, and BRIDGE



*Politicians are the same all over.
They promise to build a bridge even where there is no river.*

—Nikita Khrushchev

In the mid-1990s, I was deeply involved with the discussions that coursed through the `comp.object` newsgroup. Those of us who posted messages on that newsgroup argued furiously about different strategies of analysis and design. At one point, we decided that a concrete example would help us evaluate each other's position. So we chose a very simple design problem and proceeded to present our favorite solutions.

The design problem was extraordinarily simple. We chose to design the software that ran inside a simple table lamp. The table lamp has a switch and a light. You could ask the switch whether it was on or off, and you could tell the light to turn on or off. A nice, simple problem.

The debate raged for months. Each person demanded that his own particular style of design was superior to all the others. Some used a simple approach of just a switch and light object. Others thought there ought to be a lamp object that contained the switch and the light. Still others thought that electricity should be an object. One person actually suggested a power-cord object.

Despite the absurdity of most of those arguments, the design model is actually interesting to explore. Consider Figure 25-1. We can certainly make this design work. The `Switch` object can poll the state of the actual switch and can send appropriate `turnOn` and `turnOff` messages to the `Light`.

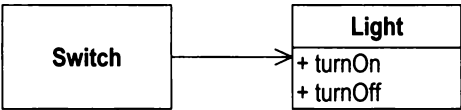


Figure 25-1 Simple table lamp

What don't we like about this design?

Two of our design principles are being violated by this design: the Dependency-Inversion Principle (DIP) and the Open-Closed Principle (OCP). The violation of the DIP is easy to see, the dependency from Switch to Light is a dependency upon a concrete class. DIP tells us to prefer dependencies on abstract classes. The violation of OCP is a little less direct, but is more to the point. We don't like this design because it forces us to drag a Light along everywhere we need a Switch. Switch cannot be easily extended to control objects other than Light.

ABSTRACT SERVER

You might be thinking that you could inherit a subclass from Switch that would control something other than a light as in Figure 25-3. But this doesn't solve the problem because FanSwitch still inherits the dependency upon Light. Wherever you take a FanSwitch, you'll have to bring Light along. In any case, that particular inheritance relationship also violates the DIP.

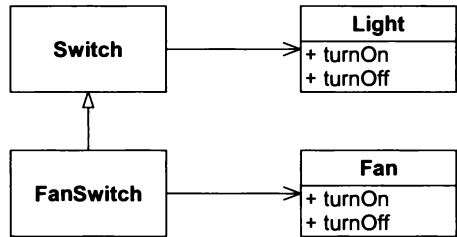


Figure 25-2 A bad way to extended Switch

To solve the problem, we invoke one of the simplest of all design patterns: ABSTRACT SERVER. (See Figure 25-2.) By introducing an interface between the Switch and the Light, we have made it possible for Switch to control anything that implements that interface. This immediately satisfies both the DIP and the OCP.

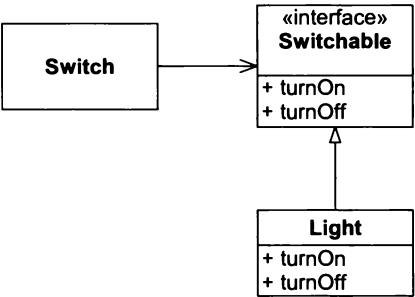


Figure 25-3 ABSTRACT SERVER solution to the Table Lamp problem

Who Owns the Interface?

As an interesting aside, notice that the interface is named for its client. It is called Switchable rather than ILight. We've talked about this before, and we'll probably notice it again. Interfaces belong to the client, not to the derivative. The logical binding between the client and the interface is stronger than the logical binding between

the interface and its derivatives. It is so strong that it makes no sense to deploy `Switch` without `Switchable`; yet it makes perfect sense to deploy `Switchable` without `Light`. The strength of the logical bonds is at odds with the strength of the physical bonds. Inheritance is a much stronger physical bond than association.

In the early 1990s, we used to think that the physical bond ruled. There were very reputable books that recommended that inheritance hierarchies be placed together in the same physical package. This seemed to make sense because inheritance is such a strong physical bond. But over the last decade, we have learned that the physical strength of inheritance is misleading and that inheritance hierarchies should usually not be packaged together. Rather, clients tend to be packaged with the interfaces they control.

This misalignment of the strength of logical and physical bonds is an artifact of statically typed languages like C++ and Java. Dynamically typed languages, like Smalltalk, Python, and Ruby, don't have the misalignment because they don't use inheritance to achieve polymorphic behavior.

Adapter

There is a problem with the design in Figure 25-3. There is a potential violation of the Single-Responsibility Principle (SRP). We have bound together two things, `Light` and `Switchable`, that may not change for the same reasons. What if we can't add the inheritance relationship to `Light`? What if we purchased `Light` from a third party and we don't have the source code. Or what if there is some other class that we want a `Switch` to control but that we can't derive from `Switchable`? Enter the ADAPTER.¹

Figure 25-4 shows how the Adapter pattern can be employed to solve the problem. The adapter derives from `Switchable` and delegates to `Light`. This solves the problem neatly. Now we can have any object that can be turned on or off controlled by a `Switch`. All we need to do is create the appropriate adapter. Indeed, the object need not even have the same `turnOn` and `turnOff` methods that `Switchable` has. The adapter can be *adapted* to the interface of the object.

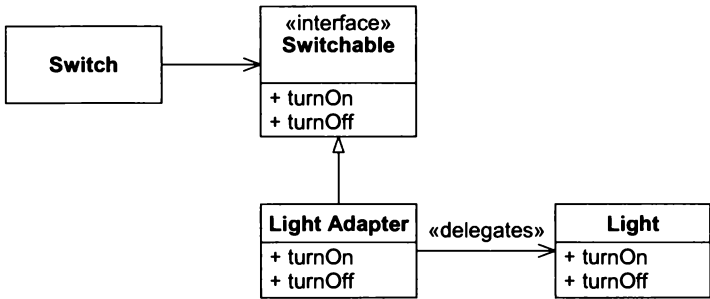


Figure 25-4 Solving the Table Lamp with ADAPTER

TANSTAAFL. Adapters don't come cheap. You need to write the new class, and you need to instantiate the adapter and bind the adapted object to it. Then, every time you invoke the adapter, you have to pay for the time and space required for the delegation. So clearly, you don't want to use adapters all the time. The ABSTRACT SERVER solution is quite appropriate for most situations. In fact, even the initial solution in Figure 25-1 is pretty good unless you happen to *know* that there are other objects for `Switch` to control.

The Class Form of ADAPTER

The `LightAdapter` class in Figure 25-4 is known as an *object form adapter*. There is another approach known as the *class form adapter*, which is shown in Figure 25-5. In this form, the adapter object inherits from both the `Switchable` interface and the `Light` class. This form is a tiny bit more efficient than the object form and is a bit easier to use, but at the expense of using the high coupling of inheritance.

1. We've seen the ADAPTER before, back in Figures 10-2 and 10-3 starting on page 118.

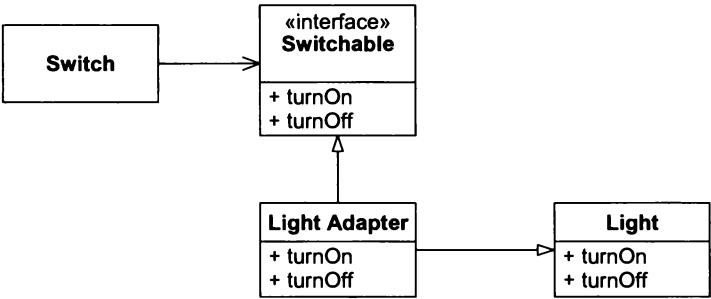


Figure 25-5 Solving the Table Lamp with ADAPTER

The Modem Problem, ADAPTERS and LSP

Consider the situation in Figure 25-6. We have a large number of modem clients all making use of the Modem interface. The Modem interface is implemented by several derivatives, including HayesModem, USRoboticsModem, and EarniesModem. This is a pretty common situation. It conforms nicely to the OCP, LSP, and DIP. Modem clients are unaffected when there are new kinds of modems to deal with. Suppose this situation were to continue for several years. Suppose that there were hundreds of modem clients all making happy use of the Modem interface.

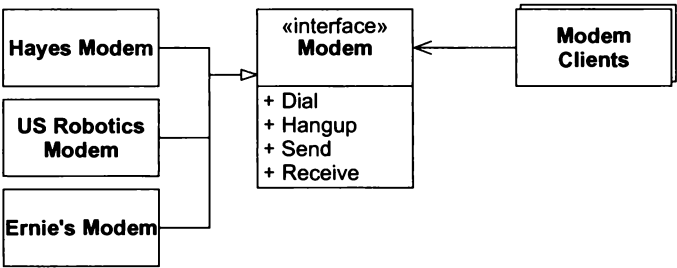


Figure 25-6 Modem Problem

Now suppose that our customers have given us a new requirement. There are certain kinds of modems that don't dial. These are called dedicated modems because they sit at both ends of a dedicated connection.² There are several new applications that use these dedicated modems and don't bother to dial. We'll call these the DedUsers. However, our customers want all the current modem clients to be able to use these dedicated modems. They tell us that they don't want to have to modify the hundreds of modem client applications, so those modem clients will simply be told to dial dummy phone numbers.

If we had our druthers, we might want to alter the design of our system as shown in Figure 25-7. We'd make use of the ISP to split the dialling and communications functions into two separate interfaces. The old modems would implement both interfaces, and the modem clients would use both interfaces. The DedUsers would use nothing but the Modem interface, and the DedicatedModem would implement just the Modem interface. Unfortunately this requires us to make changes to all the modem clients—something that our customers forbade.

So what do we do? We can't separate the interfaces as we'd like, yet we must provide a way for all the modem clients to use DedicatedModem. One possible solution is to derive DedicatedModem from Modem and to implement the dial and hangup functions to do nothing, as follows:

2. All modems used to be dedicated. It is only in recent geological epochs that modems took on the ability to dial. In the early Jurassic period, you rented a breadbox-sized modem from the phone company and connected it to another modem through dedicated lines that you also rented from the phone company (life was good for the phone company in the Jurassic). If you wanted to dial, you rented another bread-box-sized unit called an auto dialer.

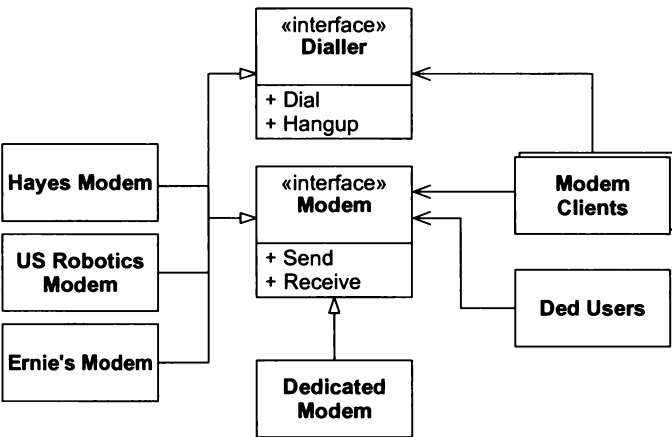


Figure 25-7 Ideal solution to the Modem Problem

```
class DedicatedModem public : Modem
{
    public:
        virtual void dial(char phoneNumber[10]) {}
        virtual void hangup() {}
        virtual void send(char c)
        {...}
        virtual char receive()
        {...}
};
```

Degenerate functions are a sign that we may be violating the LSP. The users of the base class may be expecting dial and hangup to significantly change the state of the modem. The degenerate implementations in DedicatedModem may violate those expectations.

Let's presume that the modem clients were written to expect their modems to be dormant until dial is called and to return to dormancy when hangup is called. In other words, they don't expect any characters to be coming out of modems that aren't dialled. DedicatedModem violates this expectation. It will return characters before dial has been called, and it will continue to return them after hangup has been called. Thus, DedicatedModem may crash some of the modem clients.

Now you might suggest that the problem is with the modem clients. They aren't written very well if they crash on unexpected input. I'd agree with that. But it's going to be hard to convince the folks who have to maintain the modem clients to make changes to their software because we are adding a new kind of modem. Not only does this violate the OCP, it's also just plain frustrating. And besides, our customer has explicitly forbidden us from changing the modem clients.

We Can Fix this with a Kludge. We can simulate a connection status in the dial and hangup methods of DedicatedModem. We can refuse to return characters if dial has not been called, or after hangup has been called. If we make this change, then all the modem clients will be happy and won't have to change. *All we have to do is convince the DedUsers to call dial and hangup.* (See Figure 25-8.)

You might imagine that the folks who are building the DedUsers find this pretty frustrating. They are explicitly using DedicatedModem. *Why should they have to call dial and hangup?* However, they haven't written their software yet, so it's easier to get them to do what we want.

A Tangled Web of Dependencies. Months later, when there are hundreds of DedUsers, our customers present us with a new change. It seems that all these years our programs have not had to dial international phone

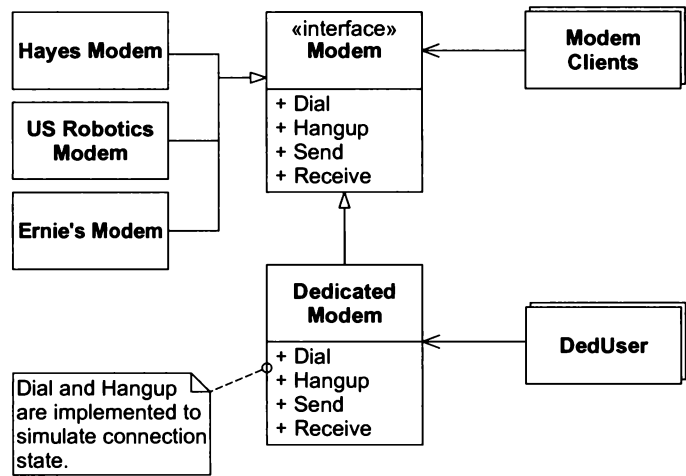


Figure 25-8 Solving the Modem Problem by kluding DedicateModem to simulate connection state

numbers. That’s why they got away with the `char[10]` in `dial`. Now, however, our customers want us to be able to dial phone numbers of arbitrary length. They have a need to make international calls, credit-card calls, PIN-identified calls, etc.

Clearly all the modem clients must be changed. They were written to expect `char[10]` for the phone number. Our customers authorize this because they have no choice, and hordes of programmers are put to the task. Just as clearly, the classes in the modem hierarchy must change to accommodate the new phone number size. Our little team can deal with that. *Unfortunately, we now have to go to the authors of the DedUsers and tell them that they have to change their code!* You might imagine how happy they’ll be about that. They aren’t calling `dial` because they need to. They are calling `dial` because we told them they have to. And now they are going through an expensive maintenance job because they did what we told them to do.

This is the kind of nasty dependency tangle that many projects find themselves in. A kludge in one part of the system creates a nasty thread of dependency that eventually causes problems in what ought to be a completely unrelated part of the system.

ADAPTER to the Rescue. We could have prevented this fiasco by using an ADAPTER to solve the initial problem as shown in Figure 25-9. In this case, `DedicatedModem` does not inherit from `Modem`. The modem clients use `DedicatedModem` indirectly through the `DedicatedModemAdapter`. This adapter implements `dial` and `hangup` to simulate the connection state. It delegates `send` and `recieve` calls to the `DedicatedModem`.

Note that this eliminates all the difficulties we had before. Modem clients are seeing the connection behavior that they expect, and `DedUsers` don’t have to fiddle with `dial` or `hangup`. When the phone number requirement changes, the `DedUsers` will be unaffected. Thus, we have fixed both the LSP and OCP violations by putting the adapter in place.

Note that the kludge still exists. The adapter is still simulating connection state. You may think this is ugly, and I’d certainly agree with you. However, notice that all the dependencies point *away* from the adapter. The kludge is isolated from the system, tucked away in an adapter that barely anybody knows about. The only hard dependency upon that adapter will likely be in the implementation of some factory somewhere.³

BRIDGE

There is another way to look at this problem. The need for a dedicated modem has added a new degree of freedom to the `Modem` type hierarchy. When the `Modem` type was initially conceived, it was simply an interface for a set of

3. See Chapter 21, `FACTORY`.

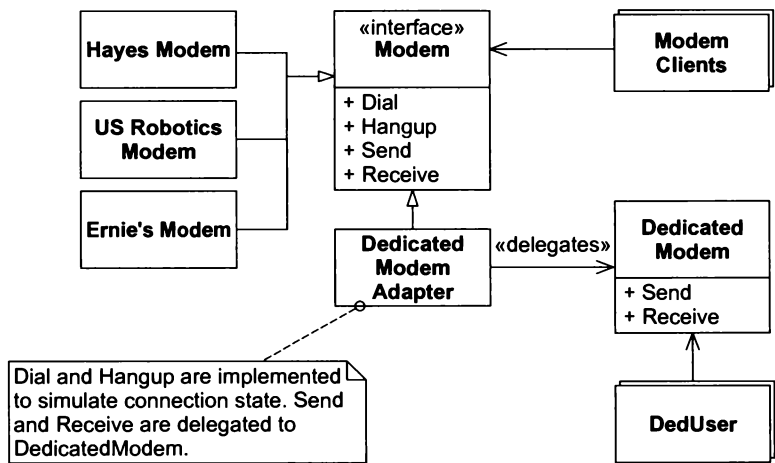


Figure 25-9 Solving the Modem Problem with the ADAPTER

different hardware devices. Thus we had HayesModem, USRModem, and ErniesModem deriving from the base Modem class. Now, however, it appears that there is another way to cut at the Modem hierarchy. We could have DialModem and DedicatedModem deriving from Modem.

Merging these two independent hierarchies can be done as shown in Figure 25-10. Each of the leaves of the type hierarchy puts either a dialup or dedicated behavior onto the hardware it controls. A DedicatedHayesModem object controls a Hayes modem in a dedicated context.

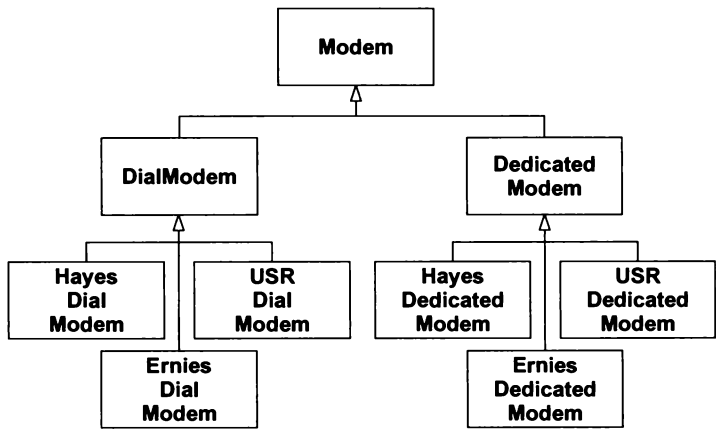


Figure 25-10 Solving the Modem Problem by merging type hierarchies

This is not an ideal structure. Every time we add a new piece of hardware, we must create *two* new classes—one for the dedicated case and one for the dialup case. Every time we add a new connection type, we have to create *three* new classes, one for each of the different pieces of hardware. If these two degrees of freedom are at all volatile, we could wind up with a large number of derived classes before too long.

The BRIDGE pattern often helps in situations where a type hierarchy has more than one degree of freedom. Rather than merge the hierarchies, we can separate them and tie them together with a bridge.

Figure 25-11 shows the structure. We split the modem hierarchy into two hierarchies. One represents the connection method, and the other represents the hardware.

Modem users continue to use the Modem interface. ModemConnectionController implements the Modem interface. The derivatives of ModemConnectionController control the connection mechanism.

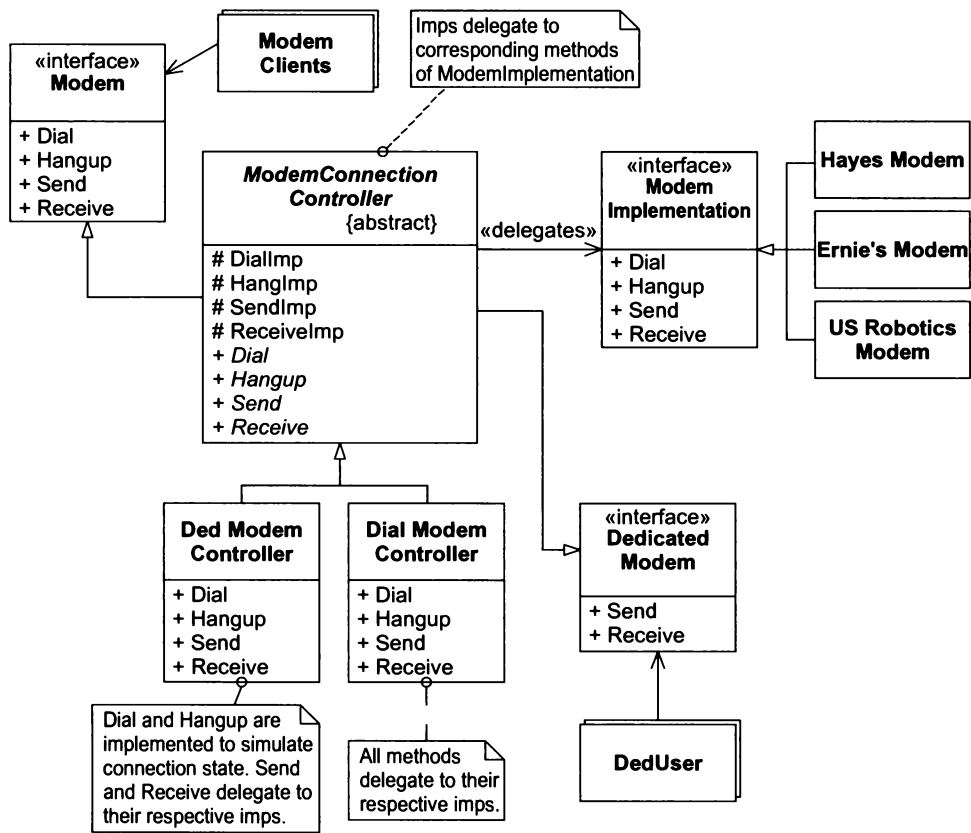


Figure 25-11 BRIDGE solution to the Modem Problem

DialModemController simply passes the dial and hangup method to dialImp and hangImp in the ModemConnectionController base class. Those methods then delegate to the ModemImplementation class where they are deployed to the appropriate hardware controller. DedModemController implements dial and hangup to simulate the connection state. It passes send and receive to sendImp and receiveImp, which are then delegated to the ModemImplementation hierarchy as before.

Note that the four imp functions in the ModemConnectionController base class are protected. This is because they are strictly to be used by derivatives of ModemConnectionController. No one else should be calling them.

This structure is complex, but interesting. We are able to create it without affecting the modem users, and yet it allows us to completely separate the connection policies from the hardware implementation. Each derivative of ModemConnectionController represents a new connection policy. That policy can use sendImp, receiveImp, dialImp, and hangImp to implement that policy. New imp functions could be created without affecting the users. The ISP could be employed to add new interfaces to the connection controller classes. This could create a migration path that the modem clients could slowly follow toward an API that is higher level than dial and hangup.

Conclusion

One might be tempted to suggest that the real problem with the Modem scenario is that the original designers got the design wrong. They should have known that connection and communication were separate concepts. Had they done a little more analysis, they would have found this and corrected it. So it is tempting to blame the problem on insufficient analysis.

Poppycock! There is no such thing as *enough* analysis. No matter how much time you spend trying to figure out the perfect software structure, you will always find that the customer will introduce a change that violates that structure.

There is no escape from this. There are no perfect structures. There are only structures that try to balance the current costs and benefits. Over time those structures must change as the requirements of the system change. The trick to managing that change is to keep the system as simple and flexible as possible.

The ADAPTER solution is simple and direct. It keeps all the dependencies pointing in the right direction, and it's very simple to implement. The BRIDGE solution is quite a bit more complex. I would not suggest embarking down that road until you had very strong evidence that you needed to completely separate the connection and communication policies and that you needed to add new connection policies.

The lesson here, as always, is that a pattern is something that comes with both costs and benefits. You should find yourself using the ones that best fit the problem at hand.

Bibliography

1. Gamma, et al. *Design Patterns*, Reading, MA: Addison-Wesley, 1995.

PROXY and STAIRWAY TO HEAVEN: Managing Third Party APIs



Does anybody remember laughter?

—Robert Plant, *The Song Remains the Same*

There are many barriers in software systems. When we move data from our program into the database, we are crossing the database barrier. When we send a message from one computer to another, we are crossing the network barrier.

Crossing these barriers can be complicated. If we aren't careful, our software will be more about the barriers than about the problem to be solved. The patterns in this chapter help us cross such barriers while keeping the program centered on the problem to be solved.

PROXY

Imagine that we are writing a shopping-cart system for a Web site. Such a system might have objects for the customer, the order (the cart), and the products in the order. Figure 26-1 shows a possible structure. This structure is simplistic, but will serve for our purposes.

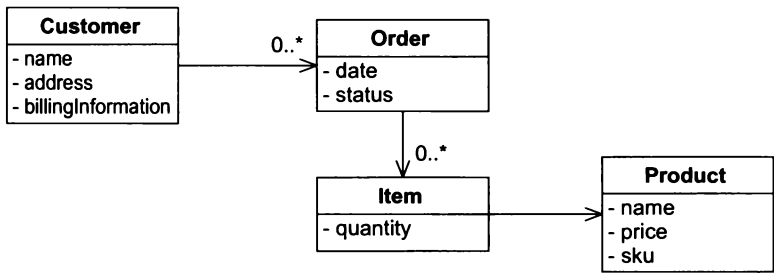


Figure 26-1 Simple shopping cart object model

If we consider the problem of adding a new item to an order, we might come up with the code in Listing 26-1. The addItem method of class Order simply creates a new Item holding the appropriate Product and quantity. It then adds that Item to its internal Vector of Items.

Listing 26-1

Adding an item to the Object Model.

```
public class Order
{
    private Vector itsItems = new Vector();
    public void addItem(Product p, int qty)
    {
        Item item = new Item(p, qty);
        itsItems.add(item);
    }
}
```

Now imagine that these objects represent data that are kept in a relational database. Figure 26-2 shows the tables and keys that might represent the objects. To find the orders for a given customer, you find all orders that have the customer’s cusid. To find all the items in a given order, you find the items that have the order’s orderId. To find the products referenced by the items, you use the product’s sku.

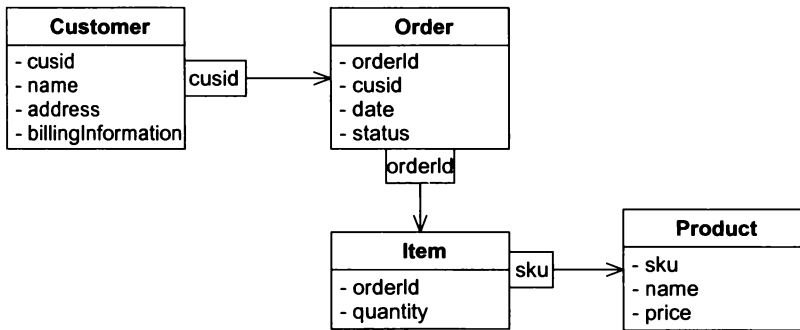


Figure 26-2 Shopping Card Relational Data Model

If we want to add an item row for a particular order, we’d use something like Listing 26-2. This code makes JDBC calls to directly manipulate the relational-data model.

Listing 26-2**Adding an item to the relational model.**

```
public class AddItemTransaction extends Transaction
{
    public void addItem(int orderId, String sku, int qty)
    {
        Statement s = itsConnection.createStatement();
        s.executeUpdate("insert into items values(" +
                        orderId + "," + sku + "," +
                        qty + ")");
    }
}
```

These two code snippets are very different, and yet they perform the same logical function. They both connect an item to an order. The first ignores the existence of a database, and the second glories in it.

Clearly the shopping cart program is all about orders, items, and products. Unfortunately, if we use the code in Listing 26-2, we make it about SQL statements, database connections, and piecing together query strings. This is a significant violation of the SRP and possibly the CCP. Listing 26-2 mixes together two concepts that change for different reasons. It mixes the concept of the items and orders with the concept of relational schemas and SQL. If either concept must change for any reason, the other concept will be affected. Listing 26-2 also violates the DIP since the policy of the program depends upon the details of the storage mechanism.

The PROXY pattern is a way to cure these ills. To explore this, let's set up a test program that demonstrates the behavior of creating an order and calculating the total price. The salient part of this program is shown in Listing 26-3.

Listing 26-3**Test program creates order and verifies calculation of price.**

```
public void testOrderPrice()
{
    Order o = new Order("Bob");
    Product toothpaste = new Product("Toothpaste", 129);
    o.addItem(toothpaste, 1);
    assertEquals(129, o.total());
    Product mouthwash = new Product("Mouthwash", 342);
    o.addItem(mouthwash, 2);
    assertEquals(813, o.total());
}
```

The simple code that passes this test is shown in Listings 26-4 through 26-6. It makes use of the simple object model in Figure 26-1. It does not assume that there is a database anywhere. It is also incomplete in many ways. It is just enough code to get the test to pass.

Listing 26-4**order.java**

```
public class Order
{
    private Vector itsItems = new Vector();
```

```

public Order(String cusid)
{
}

public void addItem(Product p, int qty)
{
    Item item = new Item(p,qty);
    itsItems.add(item);
}

public int total()
{
    int total = 0;
    for (int i = 0; i < itsItems.size(); i++)
    {
        Item item = (Item) itsItems.elementAt(i);
        Product p = item.getProduct();
        int qty = item.getQuantity();
        total += p.getPrice() * qty;
    }
    return total;
}
}

```

Listing 26-5

product.java

```

public class Product
{
    private int itsPrice;

    public Product(String name, int price)
    {
        itsPrice = price;
    }

    public int getPrice()
    {
        return itsPrice;
    }
}

```

Listing 26-6

item.java

```

public class Item
{
    private Product itsProduct;
    private int itsQuantity;

    public Item(Product p, int qty)
    {
        itsProduct = p;
        itsQuantity = qty;
    }
}

```

```
public Product getProduct()  
{  
    return itsProduct;  
}  
  
public int getQuantity()  
{  
    return itsQuantity;  
}  
}
```

Figures 26-3 and 26-4 show how the PROXY pattern works. Each object that is to be proxied is split into three parts. The first is an interface that declares all the methods that clients need to invoke. The second is a class that implements those methods without knowledge of the database. The third is the proxy that knows about the database.

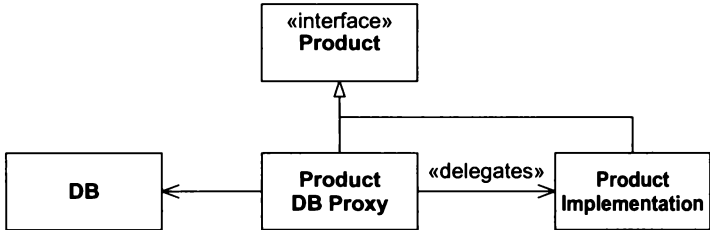


Figure 26-3 PROXY static model

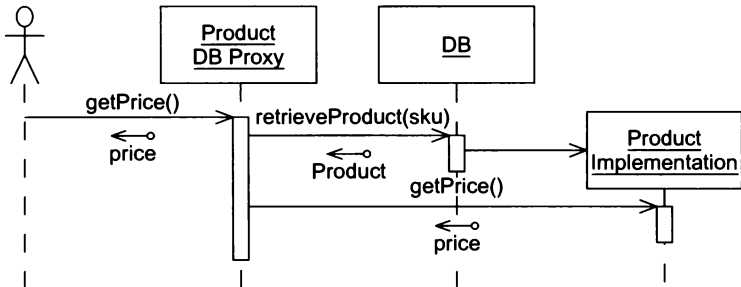


Figure 26-4 PROXY dynamic model

Consider the `Product` class. We have proxied it by replacing it with an interface. This interface has all the same methods that `Product` has. The `ProductImplementation` class implements the interface almost exactly as before. The `ProductDBProxy` implements all the methods of `Product` to fetch the product from the database, create an instance of `ProductImplementation`, and then delegate the message to it.

The sequence diagram in Figure 26-4 shows how this works. The client sends the `getPrice` message to what it thinks is a `Product`, but what is really a `ProductDBProxy`. The `ProductDBProxy` fetches the `ProductImplementation` from the database. It then delegates the `getPrice` method to it.

Neither the client nor the `ProductImplementation` knows that this has happened. The database has been inserted into the application without either party knowing about it. That’s the beauty of the PROXY pattern. In theory, it can be inserted in between two collaborating objects without those objects having to know about it. Thus, it can be used to cross a barrier like a database or a network without either of the participants knowing about it.

In reality, using proxies is nontrivial. To get an idea what some of the problems are, let’s try to add the PROXY pattern to the simple shopping cart application.

Proxifying the Shopping Cart

The simplest Proxy to create is for the `Product` class. For our purposes, the product table represents a simple dictionary. It will be loaded in one place with all the products. There is no other manipulation of this table, and that makes the proxies relatively trivial.

To get started, we need a simple database utility that stores and retrieves product data. The proxy will use this interface to manipulate the database. Listing 26-7 shows the test program for what I have in mind. Listings 26-8 and 26-9 make that test pass.

Listing 26-7

DBTest.java

```
import junit.framework.*;
import junit.swingui.TestRunner;

public class DBTest extends TestCase
{
    public static void main(String[] args)
    {
        TestRunner.main(new String[]{"DBTest"});
    }

    public DBTest(String name)
    {
        super(name);
    }

    public void setUp() throws Exception
    {
        DB.init();
    }

    public void tearDown() throws Exception
    {
        DB.close();
    }

    public void testStoreProduct() throws Exception
    {
        ProductData storedProduct = new ProductData();
        storedProduct.name = "MyProduct";
        storedProduct.price = 1234;
        storedProduct.sku = "999";
        DB.store(storedProduct);
        ProductData retrievedProduct = DB.getProductData("999");
        DB.deleteProductData("999");
        assertEquals(storedProduct, retrievedProduct);
    }
}
```

Listing 26-8

ProductData.java

```
public class ProductData
{
    public String name;
```

```
public int price;
public String sku;

public ProductData()
{
}

public ProductData(String name, int price, String sku)
{
    this.name = name;
    this.price = price;
    this.sku = sku;
}

public boolean equals(Object o)
{
    ProductData pd = (ProductData)o;
    return name.equals(pd.name) &&
        sku.equals(pd.sku) &&
        price==pd.price;
}
}
```

Listing 26-9

DB.java

```
import java.sql.*;

public class DB
{
    private static Connection con;

    public static void init() throws Exception
    {
        Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
        con = DriverManager.getConnection(
            "jdbc:odbc:PPP Shopping Cart");
    }

    public static void store(ProductData pd) throws Exception
    {
        PreparedStatement s = buildInsertionStatement(pd);
        executeStatement(s);
    }

    private static PreparedStatement
    buildInsertionStatement(ProductData pd) throws SQLException
    {
        PreparedStatement s = con.prepareStatement(
            "INSERT into Products VALUES (?, ?, ?)");
        s.setString(1, pd.sku);
        s.setString(2, pd.name);
        s.setInt(3, pd.price);
        return s;
    }
}
```

```

public static ProductData getProductData(String sku) throws Exception
{
    PreparedStatement s = buildProductQueryStatement(sku);
    ResultSet rs = executeQueryStatement(s);
    ProductData pd = extractProductDataFromResultSet(rs);
    rs.close();
    s.close();
    return pd;
}

private static PreparedStatement
buildProductQueryStatement(String sku) throws SQLException
{
    PreparedStatement s = con.prepareStatement(
        "SELECT * FROM Products WHERE sku = ?;");
    s.setString(1, sku);
    return s;
}

private static ProductData
extractProductDataFromResultSet(ResultSet rs) throws SQLException
{
    ProductData pd = new ProductData();
    pd.sku = rs.getString(1);
    pd.name = rs.getString(2);
    pd.price = rs.getInt(3);
    return pd;
}

public static void deleteProductData(String sku) throws Exception
{
    executeStatement(buildProductDeleteStatement(sku));
}

private static PreparedStatement
buildProductDeleteStatement(String sku) throws SQLException
{
    PreparedStatement s = con.prepareStatement(
        "DELETE from Products where sku = ?");
    s.setString(1, sku);
    return s;
}

private static void executeStatement(PreparedStatement s) throws SQLException
{
    s.execute();
    s.close();
}

private static ResultSet
executeQueryStatement(PreparedStatement s)

```

```

throws SQLException
{
    ResultSet rs = s.executeQuery();
    rs.next();
    return rs;
}

public static void close() throws Exception
{
    con.close();
}
}

```

The next step in implementing the proxy is to write a test that shows how it works. This test adds a product to the database. It then creates a `ProductProxy` with the `sku` of the stored product and attempts to use the accessors of `Product` to acquire the data from the proxy. (See Listing 26-10.)

Listing 26-10

ProxyTest.java

```

import junit.framework.*;
import junit.swingui.TestRunner;

public class ProxyTest extends TestCase
{
    public static void main(String[] args)
    {
        TestRunner.main(new String[]{"ProxyTest"});
    }

    public ProxyTest(String name)
    {
        super(name);
    }

    public void setUp() throws Exception
    {
        DB.init();
        ProductData pd = new ProductData();
        pd.sku = "ProxyTest1";
        pd.name = "ProxyTestName1";
        pd.price = 456;
        DB.store(pd);
    }

    public void tearDown() throws Exception
    {
        DB.deleteProductData("ProxyTest1");
        DB.close();
    }

    public void testProductProxy() throws Exception
    {
        Product p = new ProductProxy("ProxyTest1");
        assertEquals(456, p.getPrice());
    }
}

```